

Alight International Multi-Vendor Marketplace

Overview

Alight International Multi-Vendor Marketplace is a high-performance, production-oriented e-commerce platform designed for multi-vendor retail operations.

Architecture

The application is designed as a **Modular Monolith**:

- **Backend:** Java 21 LTS, Spring Boot 3.3.x, Spring Security, Spring Data JPA, Flyway migrations, PostgreSQL.
- **Frontend:** Next.js (App Router), React, TypeScript, Tailwind CSS, Centralized API Client.
- **API Standard:** RESTful `/api/v1/` endpoints with unified JSON responses and error structures.

Project Structure

```
TEXT

alight.com/
├── backend/                                # Spring Boot 3.x Modular Monolith
│   ├── pom.xml
│   └── src/
│       ├── main/java/com/alight/marketplace/
│       │   ├── AlightMarketplaceApplication.java
│       │   ├── common/                    # Responses, errors, validations, constants
│       │   ├── config/                   # Security, CORS, OpenAPI documentation
│       │   └── modules/                  # Domain modules (auth, vendor, product, etc.)
│       └── main/resources/
│           ├── application.yml
│           └── db/migration/              # Flyway SQL migrations
├── frontend/                              # Next.js TypeScript Frontend
│   ├── src/
│   │   ├── app/
│   │   │   ├── (storefront)/             # Customer shopping shell
│   │   │   ├── (vendor)/                 # Vendor portal shell
│   │   │   └── (admin)/                  # Admin operational shell
│   │   ├── components/                   # Reusable UI & Layout components
│   │   ├── services/                     # Centralized typed API clients
│   │   └── styles/                       # Brand design tokens & Tailwind config
├── docker/                               # Database & container configs
├── docker-compose.yml
├── .env.example
└── README.md
```

Prerequisites

- Java 21 LTS or newer

- **Maven 3.9+**
- **Node.js 20+ & npm 10+**
- **PostgreSQL 15+ or Docker**

Getting Started

1. Environment Configuration

Copy `.env.example` to `.env` in the root:

BASH

```
cp .env.example .env
```

2. Start Database

Using Docker Compose:

BASH

```
docker compose up -d postgres
```

Or use a local PostgreSQL instance listening on port `5432` with database `alight_db`.

3. Start Backend

BASH

```
cd backend  
mvn clean spring-boot:run
```

- Health Check: `GET http://localhost:8080/api/v1/health`
- OpenAPI Swagger UI: `http://localhost:8080/swagger-ui.html`
- API Documentation: `http://localhost:8080/v3/api-docs`

4. Start Frontend

BASH

```
cd frontend  
npm install  
npm run dev
```

- Customer Storefront: `http://localhost:3000`
- Vendor Portal: `http://localhost:3000/vendor`
- Admin Portal: `http://localhost:3000/admin`

Development Stages (20-Stage Architecture)

- **Stage 1: Project Foundation** (Production-Grade: Live DB diagnostics, MDC correlation IDs, security

headers, resilient API client with timeout & auto-token fallback, branded 404/500/loading error boundaries)

- **Stage 2: Database / ERD** (71 tables, Flyway migrations V1-V30, PostgreSQL 18, automated updated_at triggers, pg_trgm search GIN indexes, @Version optimistic locking)
- **Stage 3: RBAC & Permission Matrix** (4-Tier Roles: Customer, Vendor, Admin, Super Admin; 45 granular permissions, JWT embedded permissions, ResourceSecurityService multi-tenant isolation, Admin RBAC management APIs)
- **Stage 4: API Specification & Documentation** (Production OpenAPI 3.0, Multi-Environment Servers, BearerAuth JWT, 6 Grouped Swagger UI Modules, STAGE_4_API_SPECIFICATION.md manual on [/swagger-ui.html](#))
- **Stage 5: Backend Modular Monolith** (Spring Boot 3.3.4, Java 21 LTS, 28 Domain Modules, In-Memory Domain Event Bus, ThreadPoolTaskExecutor with MDC Correlation, PagedResponse & 409 Conflict Handling)
- **Stage 6: Authentication & Session Management** (JWT, refresh tokens, BCrypt hashing)
- **Stage 7: Vendor Management** (Registration, KYC verification, Admin approvals, status transitions)
- **Stage 8: Product Catalog** (Modular kitchen, wardrobe, bathroom hardware; hierarchical categories, variants)
- **Stage 9: Inventory** (Warehouse stock tracking, reservation engine, oversell prevention)
- **Stage 10: Shopping Cart** (Multi-vendor cart items, currency conversions)
- **Stage 11: Multi-Vendor Orders** (Parent order #10001 → Vendor fulfillment orders #10001-A, #10001-B)
- **Stage 12: Payments** (Razorpay webhook verification, signature security, COD threshold validation)
- **Stage 13: Logistics & Shipping** (Multi-carrier abstraction, zone/weight matrix, packing slips, tracking)
- **Stage 14: Returns, Refunds & RTO** (7-day window, photo verification, reverse logistics, RTO separation)
- **Stage 15: Settlements & Commission** (Configurable commissions, ledger calculation, hold periods, payouts)
- **Stage 16: Admin Dashboard & Moderation** (Vendor approvals, catalog control, disputes, settlement ledger)
- **Stage 17: Vendor Dashboard & Store Portal** (Store profile, product management, fulfillment, earnings)
- **Stage 18: Customer Marketplace Experience** (Modern storefront, instant search with facets, multi-step checkout)
- **Stage 19: Security Hardening & Audit Logging** (Immutable audit_logs table, transaction-isolated recording)
- **Stage 20: Containerized Production Deployment** (Multi-stage Dockerfiles, docker-compose.prod.yml)*

Stage 2: Production-Grade Database & ERD Specification

Alight International Multi-Vendor Marketplace Document Version: 2.0.0 (Production-Ready) Target Engine: PostgreSQL 16+ / PostgreSQL 18 Migration Framework: Flyway (`V1__initial_schema.sql` through `V30__production_erd_optimizations_and_triggers.sql`) Total Tables: 71 Base Tables + 6 System Views/Metadata

1. Architectural Principles & Enterprise Standards

- 1. **Primary Key Standard:** All business tables use cryptographically secure 128-bit UUIDs (`gen_random_uuid()`) to prevent enumeration attacks, enable client-side UUID pre-generation, and support distributed multi-warehouse synchronization.
- 2. **Optimistic Locking:** Critical transactional entities (`warehouse_stock` , `vendor_wallets` , `orders` , `vendor_orders` , `products` , `carts` , `coupons`) feature a `version BIGINT DEFAULT 0` column checked via JPA `@Version` to eliminate race conditions during high-volume flash sales and concurrent checkouts.
- 3. **Automated Timestamp Triggers:** High-performance PL/pgSQL trigger (`trg_set_updated_at`) automatically updates `updated_at` on every row mutation, eliminating application-side timestamp drift.
- 4. **Trigram & Full-Text Search:** PostgreSQL extension `pg_trgm` powers GIN indexes across product titles, SKUs, brand names, and vendor store names for sub-10ms autocomplete, typo tolerance, and faceted catalog search.
- 5. **Multi-Vendor Partitioning:** Orders are architected with a Parent-Child pattern (`orders` → `vendor_orders` → `order_items`) enabling independent vendor shipping, partial fulfillment, vendor-isolated settlements, and RMA handling.

2. Global Entity Table Dictionary (71 Tables)

Domain	Table Name	PK Type	Description	Concurrency
System	system_metadata	VARCHAR(64)	System-level health markers and migration flags	-
User & Auth	users	UUID	Customer, vendor, and administrator credentials	updated_at
User & Auth	roles	UUID	RBAC roles (CUSTOMER, VENDOR, ADMIN, SUPER_ADMIN)	Immutable
User & Auth	permissions	UUID	Fine-grained API and resource permissions	Immutable
User & Auth	user_roles	(user_id, role_id)	Cross-table mapping users to roles	-
User & Auth	role_permissions	(role_id, perm_id)	Cross-table mapping roles to permissions	-
User & Auth	user_addresses	UUID	Customer billing and shipping delivery addresses	updated_at
User & Auth	user_sessions	UUID	Active user sessions and device telemetry	-
User & Auth	refresh_tokens	UUID	JWT refresh tokens with rotation and revocation	-
Vendor	vendors	UUID	Multi-vendor business entity and verification state	updated_at
Vendor	vendor_business_details	UUID	Legal registration, GSTIN, PAN, and KYC details	updated_at
Vendor	vendor_pickup_addresses	UUID	Warehouse pickup locations for carrier dispatches	updated_at

Domain	Table Name	PK Type	Description	Concurrency
Vendor	vendor_documents	UUID	KYC documents, GST certificates, and identity proofs	-
Vendor	vendor_bank_accounts	UUID	Verified bank accounts for automated payout disbursements	updated_at
Catalog	categories	UUID	Hierarchical N-level product category tree	updated_at
Catalog	brands	UUID	Official manufacturer and designer brands	updated_at
Catalog	products	UUID	Core product master record with pricing and SEO	@Version
Catalog	product_variants	UUID	Size, finish, color, and dimension variations	updated_at
Catalog	product_images	UUID	High-res gallery media, CDN URLs, and alt texts	-
Catalog	product_attributes	UUID	Key-value technical specifications (Material, Grade)	-
Catalog	product_tags	UUID	Search tags and promotional taxonomy	-
Inventory	warehouses	UUID	Regional fulfillment hubs and vendor warehouses	updated_at
Inventory	warehouse_stock	UUID	SKU stock on-hand, reserved, and safety thresholds	@Version
Inventory	stock_reservations	UUID	Temporary cart/checkout stock locks with TTL	-
Inventory	stock_transactions	UUID	Immutable ledger of stock movements (IN/OUT/ADJ)	Immutable

Domain	Table Name	PK Type	Description	Concurrency
Inventory	inventory_audit_logs	UUID	Physical stock count reconciliation logs	Immutable
Pricing & Tax	currencies	UUID	Supported marketplace currencies (INR, USD, AED, EUR)	updated_at
Pricing & Tax	currency_exchange_rates	UUID	Real-time and manual FX conversion rates	updated_at
Pricing & Tax	tax_categories	UUID	Tax classes (Hardware 18%, Brass Craft 12%, etc.)	updated_at
Pricing & Tax	tax_jurisdictions	UUID	State/Regional tax rules (Inter-state IGST vs CGST/SGST)	updated_at
Pricing & Tax	tax_rules	UUID	Jurisdiction-to-tax category rate mappings	updated_at
Pricing & Tax	tax_rate_components	UUID	Tax breakdowns (CGST 9%, SGST 9%, IGST 18%)	updated_at
Pricing & Tax	product_tier_prices	UUID	B2B bulk purchase tiered discount pricing matrix	updated_at
Cart	carts	UUID	Customer and guest active shopping carts	@Version
Cart	cart_items	UUID	Multi-vendor cart line items and selections	updated_at
Orders	orders	UUID	Master customer order with grand total and payment state	@Version
Orders	vendor_orders	UUID	Split sub-orders routed to specific vendors	@Version
Orders	order_items	UUID	Granular purchased product variant line items	-

Domain	Table Name	PK Type	Description	Concurrency
Orders	order_addresses	UUID	Immutable delivery and billing address snapshots	-
Orders	order_status_history	UUID	Audit trail of all order workflow state transitions	Immutable
Payment	payment_transactions	UUID	Razorpay / Stripe payment gateway attempt logs	updated_at
Payment	payment_webhook_logs	UUID	Idempotent gateway webhook payload logs	Immutable
Settlement	vendor_wallets	UUID	Vendor available balance, pending escrow, and earnings	@Version
Settlement	wallet_transactions	UUID	Double-entry ledger of credits, debits, and holds	Immutable
Settlement	refund_transactions	UUID	Customer refund disbursements and reverse charges	-
Settlement	escrow_ledgers	UUID	Order hold periods before final vendor release	-
Logistics	shipping_carriers	UUID	Integrated courier partners (Delhivery, BlueDart, Blr)	updated_at
Logistics	shipping_pincode_zones	UUID	Regional pincode zones (Metro, Tier-1, Remote)	updated_at
Logistics	shipping_rate_rules	UUID	Weight and volumetric slab pricing matrix	updated_at
Logistics	shipment_packages	UUID	Physical dispatch boxes, AWB numbers, and dimensions	updated_at
Logistics	shipment_tracking_events	UUID	Carrier tracking checkpoint timeline	Immutable
Logistics	shipping_manifests	UUID	Bulk carrier pickup manifests and	-

Domain	Table Name	PK Type	Description	Concurrency
			handoff sheets	
Returns & RMA	rma_requests	UUID	Customer return/exchange claims (7-day window)	updated_at
Returns & RMA	rma_items	UUID	Specific items claimed with return reason and proof	-
Returns & RMA	rma_events	UUID	Return inspection, approval, and logistics lifecycle	Immutable
Returns & RMA	rma_policies	UUID	Vendor and category return window rules	updated_at
Returns & RMA	rma_inspections	UUID	Physical warehouse return grading reports	-
Promotions	coupons	UUID	Global and vendor-scoped discount vouchers	@Version
Promotions	coupon_usages	UUID	User coupon redemption tracking and limits	-
Promotions	coupon_categories	UUID	Category whitelist/blacklist restrictions	-
Promotions	promotion_rules	UUID	Dynamic pricing campaigns (e.g. Buy 2 Get 10%)	updated_at
Promotions	flash_sales	UUID	Timed marketplace flash sale events	updated_at
Promotions	flash_sale_products	UUID	Products assigned to flash sales with quotas	-
Reviews & QA	product_reviews	UUID	Customer verified purchase ratings and reviews	updated_at
Reviews & QA	review_images	UUID	Customer uploaded review photos	-
Reviews & QA	review_votes	UUID	Helpful / unhelpful upvotes on reviews	-

Domain	Table Name	PK Type	Description	Concurrency
Reviews & QA	product_questions	UUID	Pre-purchase customer Q&A threads	updated_at
Reviews & QA	product_answers	UUID	Vendor and expert verified answers	updated_at
Reviews & QA	question_votes	UUID	Upvotes on questions and answers	-
Support	support_tickets	UUID	Multi-channel customer & vendor dispute tickets	updated_at
Support	ticket_messages	UUID	Threaded discussion messages and attachments	-
Support	ticket_attachments	UUID	Uploaded dispute evidence and PDFs	-
Notifications	notification_templates	UUID	Email, SMS, and In-App notification blueprints	updated_at
Notifications	notifications	UUID	Real-time user alert inbox and read statuses	updated_at
Compliance	marketplace_commission_invoices	UUID	Monthly B2B GST tax invoices for vendor commissions	updated_at
Compliance	tax_compliance_ledgers	UUID	Section 52 TCS (1%) and Section 194-O TDS records	updated_at
Compliance	settlement_cycles	UUID	Weekly/Bi-weekly automated payout cycle batches	updated_at
Compliance	vendor_payouts	UUID	Bank NEFT/RTGS transaction disbursement batches	updated_at
Compliance	payout_batches	UUID	Multi-vendor consolidated payout disbursement files	updated_at

Domain	Table Name	PK Type	Description	Concurrency
Engagement	search_synonyms	UUID	Search keyword synonym maps (e.g. "telescopic" → "drawer channel")	-
Engagement	wishlists	UUID	Customer saved collections and moodboards	updated_at
Engagement	wishlist_items	UUID	Saved products inside customer wishlists	-
Engagement	product_bundles	UUID	Curated hardware kits (e.g., "Modular Kitchen Set")	updated_at
Engagement	product_bundle_items	UUID	Component products within a bundled hardware kit	-
Engagement	recently_viewed_products	UUID	Customer recent browsing history for recommendations	-
Engagement	quote_requests	UUID	B2B bulk architectural hardware Request For Quote (RFQ)	updated_at
Engagement	quote_items	UUID	Bill of Quantities (BOQ) line items for RFQ	-
Audit	audit_logs	UUID	Tamper-evident operational audit trail	Immutable

3. Domain ER Diagrams (Mermaid)

Domain 1: Multi-Vendor Catalog, Categories, Variants & Images

MERMAID

erDiagram

```
CATEGORIES ||--o{ CATEGORIES : "parent_id"
CATEGORIES ||--o{ PRODUCTS : "category_id"
BRANDS ||--o{ PRODUCTS : "brand_id"
VENDORS ||--o{ PRODUCTS : "vendor_id"
PRODUCTS ||--o{ PRODUCT_VARIANTS : "product_id"
PRODUCTS ||--o{ PRODUCT_IMAGES : "product_id"
PRODUCTS ||--o{ PRODUCT_ATTRIBUTES : "product_id"
PRODUCTS ||--o{ PRODUCT_TIER_PRICES : "product_id"
PRODUCTS ||--o{ PRODUCT_TAGS : "product_id"
```

PRODUCTS {

```
    UUID id PK
    UUID vendor_id FK
    UUID category_id FK
    UUID brand_id FK
    VARCHAR title
    VARCHAR slug UK
    NUMERIC base_price
    NUMERIC discount_price
    VARCHAR sku UK
    INTEGER stock_quantity
    VARCHAR status
    BIGINT version
    TIMESTAMP created_at
    TIMESTAMP updated_at
```

}

PRODUCT_VARIANTS {

```
    UUID id PK
    UUID product_id FK
    VARCHAR variant_name
    VARCHAR variant_sku UK
    NUMERIC price
    INTEGER stock_quantity
    BOOLEAN is_active
```

}

Domain 2: Multi-Warehouse Inventory & Real-Time Stock Engine

MERMAID

```
erDiagram
    WAREHOUSES ||--o{ WAREHOUSE_STOCK : "warehouse_id"
    PRODUCTS ||--o{ WAREHOUSE_STOCK : "product_id"
    PRODUCT_VARIANTS ||--o{ WAREHOUSE_STOCK : "variant_id"
    WAREHOUSE_STOCK ||--o{ STOCK_TRANSACTIONS : "stock_id"
    PRODUCTS ||--o{ STOCK_RESERVATIONS : "product_id"

    WAREHOUSE_STOCK {
        UUID id PK
        UUID warehouse_id FK
        UUID product_id FK
        UUID variant_id FK
        INTEGER quantity_on_hand
        INTEGER quantity_reserved
        INTEGER reorder_threshold
        INTEGER safety_stock
        BIGINT version
        TIMESTAMP updated_at
    }

    STOCK_RESERVATIONS {
        UUID id PK
        UUID product_id FK
        UUID variant_id FK
        UUID warehouse_id FK
        VARCHAR session_id
        INTEGER quantity
        VARCHAR status
        TIMESTAMP expires_at
    }
```

Domain 3: Multi-Vendor Order Splitting, Fulfillment & Escrow

MERMAID

```

erDiagram
    USERS ||--o{ ORDERS : "user_id"
    ORDERS ||--o{ VENDOR_ORDERS : "master_order_id"
    VENDORS ||--o{ VENDOR_ORDERS : "vendor_id"
    VENDOR_ORDERS ||--o{ ORDER_ITEMS : "vendor_order_id"
    ORDERS ||--o{ ORDER_ADDRESSES : "order_id"
    ORDERS ||--o{ ORDER_STATUS_HISTORY : "order_id"
    ORDERS ||--o{ PAYMENT_TRANSACTIONS : "order_id"
    VENDOR_ORDERS ||--o{ SHIPMENT_PACKAGES : "vendor_order_id"
    VENDORS ||--|| VENDOR_WALLETS : "vendor_id"
    VENDOR_WALLETS ||--o{ WALLET_TRANSACTIONS : "wallet_id"

    ORDERS {
        UUID id PK
        VARCHAR order_number UK
        UUID user_id FK
        NUMERIC grand_total
        VARCHAR order_status
        VARCHAR payment_status
        BIGINT version
        TIMESTAMP created_at
    }

    VENDOR_ORDERS {
        UUID id PK
        UUID master_order_id FK
        UUID vendor_id FK
        VARCHAR sub_order_number UK
        NUMERIC grand_total
        VARCHAR fulfillment_status
        BIGINT version
        TIMESTAMP created_at
    }

    VENDOR_WALLETS {
        UUID id PK
        UUID vendor_id FK
        NUMERIC available_balance
        NUMERIC pending_balance
        NUMERIC total_earnings
        BIGINT version
        TIMESTAMP updated_at
    }

```

4. Production Database Configuration

HikariCP High-Throughput Connection Pool

YAML

```
spring:
  datasource:
    hikari:
      pool-name: AlightHikariPool
      maximum-pool-size: 20
      minimum-idle: 5
      idle-timeout: 300000      # 5 minutes
      max-lifetime: 1200000    # 20 minutes
      connection-timeout: 20000 # 20 seconds
      leak-detection-threshold: 15000
      connection-test-query: SELECT 1
```

PostgreSQL Indexing Strategy

- **Primary Keys:** B-Tree Indexes on all UUID PKs.
- **Foreign Keys:** B-Tree Indexes on every FK column to eliminate table scan locks on cascades.
- **Search:** Trigram GIN indexes (`gin_trgm_ops`) on high-frequency textual filters.
- **Query Optimization:** Composite indexes ordered by (`filtering_column, sorting_column`).
- **Soft Filter Optimization:** Partial indexes (`WHERE status = 'ACTIVE'`) to keep memory footprint minimal.

Stage 3: Production-Grade RBAC & Permission Matrix Specification

Alight International Multi-Vendor Marketplace *Document Version:* 3.0.0 (Production-Ready) *Security Architecture:* Stateless JWT Bearer Authentication + Spring Security 6 Method Security + Multi-Tenant Resource Isolation *Database Authority:* PostgreSQL `roles` , `permissions` , `role_permissions` , `user_roles` (Flyway `V2` , `V31`)

1. 4-Tier Role Architecture

Role Identifier	Display Name	Target Persona	Scope of Authority
<code>ROLE_CUSTOMER</code>	Registered Customer	Shoppers, Architects, B2B Contractors	Personal cart, orders, addresses, reviews, Q&A, and return requests.
<code>ROLE_VENDOR</code>	Verified Vendor / Seller	Hardware manufacturers, boutique brass studios	Tenant-isolated store profile, catalog, stock, sub-orders, wallet & returns.
<code>ROLE_ADMIN</code>	Platform Operations Admin	Marketplace moderators, catalog managers	System-wide catalog moderation, vendor KYC approval, dispute resolution.
<code>ROLE_SUPER_ADMIN</code>	Super Administrator	Executive leadership, Lead DevOps	Full unrestricted access, user role mutation, financial ledger & audit trail.

2. Comprehensive 4-Tier Role-Permission Matrix (45 Granular Permissions)

Domain	Granular Permission Key	Description	Customer	Vendor	Admin	Super Admin
Profile	<code>user:profile_manage</code>	Read and update personal profile	✓	✓	✓	✓
Profile	<code>user:address_manage</code>	Manage personal shipping addresses	✓	✓	✓	✓
Catalog	<code>catalog:read</code>	Browse active catalog & categories	✓	✓	✓	✓
Catalog	<code>catalog:create</code>	Create new draft products & variants	✗	✓	✓	✓
Catalog	<code>catalog:update</code>	Edit owned product details & prices	✗	✓ (Tenant)	✓	✓
Catalog	<code>catalog:delete</code>	Archive/delete owned products	✗	✓ (Tenant)	✓	✓
Catalog	<code>catalog:approve</code>	Approve/reject vendor submissions	✗	✗	✓	✓
Catalog	<code>catalog:publish</code>	Publish categories & brand taxonomy	✗	✗	✓	✓
Inventory	<code>inventory:read</code>	View warehouse stock & reserves	✗	✓ (Tenant)	✓	✓
Inventory	<code>inventory:update</code>	Update owned stock on-hand	✗	✓ (Tenant)	✓	✓
Inventory	<code>inventory:adjust</code>	Stock reconciliation & threshold adjustments	✗	✓ (Tenant)	✓	✓
Inventory	<code>inventory:reserve</code>	Temporary checkout stock reservation	✗	✗	✓	✓
Orders	<code>order:read_own</code>	View personal placed orders	✓ (Own)	✗	✓	✓
Orders	<code>order:read_vendor</code>	View assigned vendor sub-orders	✗	✓ (Tenant)	✓	✓

Domain	Granular Permission Key	Description	Customer	Vendor	Admin	Super Admin
Orders	<code>order:read_all</code>	View all platform master & sub-orders	✗	✗	✓	✓
Orders	<code>order:update_status</code>	Update fulfillment status (SHIPPED)	✗	✓ (Tenant)	✓	✓
Orders	<code>order:cancel</code>	Cancel customer order / out-of-stock item	✓ (Own)	✗	✓	✓
Vendor	<code>vendor:profile_read</code>	View vendor business details	✗	✓ (Tenant)	✓	✓
Vendor	<code>vendor:profile_update</code>	Update store logo, description, pickup	✗	✓ (Tenant)	✓	✓
Vendor	<code>vendor:kyc_submit</code>	Submit GSTIN, PAN, bank verification	✗	✓ (Tenant)	✓	✓
Vendor	<code>vendor:verify_approve</code>	Review & approve vendor KYC	✗	✗	✓	✓
Vendor	<code>vendor:commission_update</code>	Configure custom commission rate per vendor	✗	✗	✓	✓
Settlement	<code>settlement:wallet_read</code>	View wallet balance & escrow holds	✗	✓ (Tenant)	✓	✓
Settlement	<code>settlement:payout_request</code>	Request withdrawal from wallet	✗	✓ (Tenant)	✓	✓
Settlement	<code>settlement:payout_approve</code>	Review & approve payout requests	✗	✗	✓	✓
Settlement	<code>settlement:payout_process</code>	Execute bank disbursement batches	✗	✗	✗	✓
Support	<code>support:ticket_create</code>	Open new support / dispute ticket	✓	✓	✓	✓
Support	<code>support:ticket_reply</code>	Post message in dispute discussion	✓ (Own)	✓ (Own)	✓	✓

Domain	Granular Permission Key	Description	Customer	Vendor	Admin	Super Admin
Support	<code>support:ticket_assign</code>	Route tickets to specific agents	✗	✗	✓	✓
Support	<code>support:ticket_resolve</code>	Close or escalate dispute tickets	✗	✗	✓	✓
Reviews	<code>review:create</code>	Submit verified purchase review	✓	✗	✓	✓
Reviews	<code>review:vote</code>	Upvote review helpfulness	✓	✓	✓	✓
Reviews	<code>review:moderate</code>	Moderate/delete inappropriate reviews	✗	✗	✓	✓
QA	<code>qa:ask</code>	Post pre-purchase product question	✓	✓	✓	✓
QA	<code>qa:answer</code>	Answer question for owned product	✗	✓ (Tenant)	✓	✓
QA	<code>qa:moderate</code>	Moderate inappropriate Q&A threads	✗	✗	✓	✓
Coupons	<code>coupon:create_vendor</code>	Create vendor-scoped discount voucher	✗	✓ (Tenant)	✓	✓
Coupons	<code>coupon:create_global</code>	Create marketplace-wide coupon campaign	✗	✗	✓	✓
Coupons	<code>coupon:manage_all</code>	Activate, edit, or deactivate any promo	✗	✗	✓	✓
Logistics	<code>logistics:manifest_create</code>	Generate carrier pickup manifests	✗	✓ (Tenant)	✓	✓
Logistics	<code>logistics:label_generate</code>	Generate AWB shipping labels	✗	✓ (Tenant)	✓	✓
Logistics	<code>logistics:carrier_manage</code>	Configure carriers & pincode zone rates	✗	✗	✓	✓
Returns	<code>returns:request_create</code>	Submit return/RMA with photo proof	✓ (Own)	✗	✓	✓

Domain	Granular Permission Key	Description	Customer	Vendor	Admin	Super Admin
Returns	<code>returns:vendor_action</code>	Accept, replace, or dispute return	✗	✓ (Tenant)	✓	✓
Returns	<code>returns:admin_override</code>	Force immediate customer refund	✗	✗	✓	✓
Returns	<code>returns:inspect</code>	Warehouse grading & inspection report	✗	✗	✓	✓
Analytics	<code>analytics:vendor_dashboard</code>	View seller GMV, conversions & returns	✗	✓ (Tenant)	✓	✓
Analytics	<code>analytics:platform_executive</code>	View marketplace GMV & tax ledgers	✗	✗	✓	✓
System	<code>system:user_role_manage</code>	Assign & revoke user roles/permissions	✗	✗	✗	✓
System	<code>system:audit_read</code>	Query immutable audit logs	✗	✗	✓	✓
System	<code>system:config_manage</code>	Configure currencies, taxes, gateways	✗	✗	✗	✓

3. Multi-Tenant Resource Isolation Architecture

Tenant Security Principles

1. **Zero Cross-Vendor Leakage:** A Vendor can strictly access and mutate catalog items, warehouse stock records, fulfillment sub-orders, and wallet transactions where `vendor_id = current_vendor_id`. 2. **Customer Isolation:** Customers cannot view or mutate orders, dispute tickets, or saved addresses belonging to other user accounts. 3. **Admin Escalation:** Platform Administrators and Super Administrators possess supervisory read/write permissions across all tenant boundaries.

Spring Security Evaluator (`ResourceSecurityService`)

JAVA

```
// Example Controller Enforcement
@PutMapping("/products/{id}")
@PreAuthorize("@securityService.isVendorOwnerOfProduct(#id) or hasRole('ADMIN')")
public ResponseEntity<ApiResponse<ProductDto>> updateProduct(@PathVariable UUID id, ...) { ... }

@GetMapping("/vendor/orders/{subOrderId}")
@PreAuthorize("@securityService.isVendorOwnerOfOrder(#subOrderId) or hasRole('ADMIN')")
public ResponseEntity<ApiResponse<VendorOrderDetailDto>> getVendorOrder(@PathVariable UUID subOrderId) { ... }
```

4. Custom Security Annotation Cheatsheet

Annotation	Target Role Authority	Usage Scenario
<code>@RequireSuperAdmin</code>	<code>ROLE_SUPER_ADMIN</code>	User role assignment, financial payout processing, audit deletion guards
<code>@RequireAdmin</code>	<code>ROLE_ADMIN</code> , <code>ROLE_SUPER_ADMIN</code>	Catalog moderation, KYC verification, refund overrides
<code>@RequireVendor</code>	<code>ROLE_VENDOR</code>	Vendor store management, dispatch manifests, product creation
<code>@RequireCustomer</code>	<code>ROLE_CUSTOMER</code>	Customer checkout, review submission, address management
<code>@RequireVendorOrAdmin</code>	<code>ROLE_VENDOR</code> , <code>ROLE_ADMIN</code> , <code>ROLE_SUPER_ADMIN</code>	Product catalog viewing, logistics tracking

5. JWT Stateless Token Payload Structure

JSON

```
{
  "sub": "seller@alight.com",
  "userId": "a0000000-0000-0000-0000-000000000002",
  "firstName": "Vikram",
  "lastName": "Sharma",
  "roles": ["ROLE_VENDOR", "ROLE_CUSTOMER"],
  "permissions": [
    "user:profile_manage",
    "catalog:read",
    "catalog:create",
    "catalog:update",
    "inventory:read",
    "order:read_vendor",
    "vendor:profile_read",
    "settlement:wallet_read"
  ],
  "iat": 1726135200,
  "exp": 1726221600
}
```

6. RBAC REST API Specification

1. GET /api/v1/admin/rbac/roles

- **Security:** @RequireAdmin
- **Response:** List of all 4 roles with full list of bound permission metadata.

2. GET /api/v1/admin/rbac/permissions

- **Security:** @RequireAdmin
- **Response:** Complete catalog of all 45 granular permissions categorized by domain (catalog , order , vendor , settlement , etc.).

3. GET /api/v1/admin/rbac/users/{userId}/roles

- **Security:** @RequireAdmin
- **Response:** Current roles, effective permissions, and vendor tenancy binding for target user.

4. PUT /api/v1/admin/rbac/users/{userId}/roles

- **Security:** @RequireSuperAdmin
- **Request Body:** { "roles": ["ROLE_VENDOR", "ROLE_CUSTOMER"] }
- **Response:** Updated user role profile and newly resolved permissions.

5. GET /api/v1/users/me/permissions

- **Security:** authenticated()
- **Response:** Self-introspection of authenticated caller's roles, permissions list, and vendor ID.

Stage 4: Production API Specification & Swagger Documentation

Alight International Multi-Vendor Marketplace Document Version: 4.0.0 (Production-Ready) API Standard: RESTful JSON v1.0.0 (/api/v1/) OpenAPI Specification URL: `GET /v3/api-docs` Interactive Swagger UI: `http://localhost:8080/swagger-ui.html`

1. Global API Standards

Base URLs by Environment

Environment	Base URL	Protocol
Local Development	<code>http://localhost:8080</code>	HTTP
Staging	<code>https://api-staging.alight.com</code>	HTTPS
Production	<code>https://api.alight.com</code>	HTTPS (TLS 1.3)

Authentication Header

All authenticated endpoints require an HTTP `Authorization` header containing a valid JWT access token:

HTTP

Authorization: Bearer <JWT_ACCESS_TOKEN>

Standard Unified Response Envelope (`ApiResponse<T>`)

Every API endpoint returns a deterministic JSON envelope:

JSON

```
{
  "success": true,
  "message": "Operation completed successfully",
  "data": { ... },
  "error": null,
  "timestamp": "2026-09-12T07:45:00Z",
  "traceId": "c4b9-8e21-4f10-91ab",
  "path": "/api/v1/products"
}
```

Standard Error Response Envelope

```
JSON

{
  "success": false,
  "message": "Validation failed on 1 field",
  "data": null,
  "error": {
    "code": "VALIDATION_FAILED",
    "message": "Invalid SKU format",
    "status": 400,
    "details": [
      {
        "field": "sku",
        "message": "SKU must not be empty"
      }
    ]
  },
  "timestamp": "2026-09-12T07:45:00Z",
  "traceId": "c4b9-8e21-4f10-91ab",
  "path": "/api/v1/products"
}
```

2. Complete Endpoint Catalog by Domain Group

Group 1: Authentication & Session (/api/v1/auth/)

Method	Endpoint	Auth	Description	Status Codes
POST	/api/v1/auth/register	Public	Register new customer or vendor account	201 , 400 , 409
POST	/api/v1/auth/login	Public	Authenticate user credentials & issue JWT tokens	200 , 401 , 403
POST	/api/v1/auth/refresh-token	Public	Refresh expired JWT access token via rotated refresh token	200 , 401
POST	/api/v1/auth/logout	Authenticated	Revoke active refresh tokens	200
GET	/api/v1/auth/me	Authenticated	Get current authenticated user profile	200 , 401

Group 2: User Account & RBAC Permissions

Method	Endpoint	Auth	Description	St Co
GET	/api/v1/users/profile	Authenticated	Retrieve customer personal profile	2 4
PUT	/api/v1/users/profile	Authenticated	Update personal profile details	2 4
GET	/api/v1/users/addresses	Authenticated	List user saved shipping & billing addresses	2
POST	/api/v1/users/addresses	Authenticated	Add a new shipping address	2 4
PUT	/api/v1/users/addresses/{id}/default	Authenticated	Set address as primary default	2 4
DELETE	/api/v1/users/addresses/{id}	Authenticated	Delete saved shipping address	2 4
GET	/api/v1/users/me/permissions	Authenticated	Introspect active roles, 45 permissions, and vendor ID	2
GET	/api/v1/admin/rbac/roles	ROLE_ADMIN	List all 4 roles and their bound permissions	2 4
GET	/api/v1/admin/rbac/permissions	ROLE_ADMIN	List all 45 fine-grained system permissions	2 4
GET	/api/v1/admin/rbac/users/{userId}/roles	ROLE_ADMIN	View target user's active roles	2 4

Method	Endpoint	Auth	Description	St Co
			& effective permissions	
PUT	/api/v1/admin/rbac/users/{userId}/roles	ROLE_SUPER_ADMIN	Assign or modify user roles	2 4 4
PUT	/api/v1/admin/rbac/roles/{roleName}/permissions	ROLE_SUPER_ADMIN	Update permissions assigned to a role	2 4

Group 3: Product Catalog, Taxonomy & Search

Method	Endpoint	Auth	Description	Status Codes
GET	/api/v1/categories	Public	List hierarchical category tree	200
GET	/api/v1/categories/{slug}	Public	Get category details by slug	200 , 404
GET	/api/v1/brands	Public	List active manufacturer & designer brands	200
GET	/api/v1/products	Public	Search and filter catalog (keyword, category, price, rating)	200
GET	/api/v1/products/{slug}	Public	Get complete product detail with variants, images & attributes	200 , 404
GET	/api/v1/products/featured	Public	Get curated high-rating featured products	200
POST	/api/v1/vendor/products	ROLE_VENDOR	Create draft hardware product	201 , 400
PUT	/api/v1/vendor/products/{id}	ROLE_VENDOR (Tenant)	Update owned vendor product	200 , 403 , 404
PUT	/api/v1/admin/products/{id}/status	ROLE_ADMIN	Approve or reject vendor product submission	200 , 403
GET	/api/v1/search/autocomplete	Public	Sub-10ms trigram autocomplete search suggestions	200
GET	/api/v1/bundles	Public	List curated hardware kits & modular package sets	200
POST	/api/v1/quotes	Public / Customer	Submit B2B architectural hardware RFQ with BOQ items	201 , 400

Group 4: Inventory & Warehouse Reservations

Method	Endpoint	Auth	Description	Status Codes
GET	/api/v1/inventory/products/{productId}/stock	Public	Check available stock across all warehouses	200 , 404
POST	/api/v1/inventory/reservations	Public / Customer	Create temporary checkout stock reservation lock (TTL 15m)	201 , 409
DELETE	/api/v1/inventory/reservations/{reservationId}	Public / Customer	Release reserved stock on cart cancellation	200
GET	/api/v1/vendor/warehouses	ROLE_VENDOR	List fulfillment warehouses and stock allocations	200
POST	/api/v1/vendor/inventory/adjust	ROLE_VENDOR	Reconcile stock counts & safety stock thresholds	200 , 400

Group 5: Shopping Cart, Multi-Vendor Checkout & Orders

Method	Endpoint	Auth	Description	Status Codes
GET	/api/v1/cart	Public / Session	Fetch active multi-vendor cart with subtotal & tax preview	200
POST	/api/v1/cart/items	Public / Session	Add item or variant to shopping cart	200 , 400 , 409
PUT	/api/v1/cart/items/{itemId}	Public / Session	Update item quantity in cart	200 , 404
DELETE	/api/v1/cart/items/{itemId}	Public / Session	Remove line item from cart	200 , 404
POST	/api/v1/checkout/process	Authenticated	Create multi-vendor split order (#10001 → #10001-A , #10001-B)	201 , 400
GET	/api/v1/orders	Authenticated	List customer personal order history	200
GET	/api/v1/orders/{orderNumber}	Authenticated (Owner)	Get master order summary & vendor breakdown	200 , 404
GET	/api/v1/vendor/orders	ROLE_VENDOR	List assigned vendor fulfillment sub-orders	200
PUT	/api/v1/vendor/orders/{subOrderNumber}/status	ROLE_VENDOR (Tenant)	Update sub-order fulfillment (PROCESSING, SHIPPED, DELIVERED)	200 , 403

Group 6: Payments, Webhooks & Settlements

Method	Endpoint	Auth	Description	Status Codes
POST	/api/v1/payments/create-intent	Authenticated	Initialize Razorpay / Stripe payment transaction intent	200 , 400
POST	/api/v1/payments/verify	Authenticated	Verify cryptographic payment signature	200 , 400
POST	/api/v1/payments/webhook	Public (Signature Verified)	Idempotent gateway webhook callback	200
GET	/api/v1/vendor/wallet	ROLE_VENDOR (Tenant)	Get vendor wallet balance, pending escrow & earnings	200
POST	/api/v1/vendor/wallet/payouts	ROLE_VENDOR	Request payout withdrawal to verified bank account	201 , 400
GET	/api/v1/admin/settlements/cycles	ROLE_ADMIN	View settlement cycles and payout batches	200 , 403
POST	/api/v1/admin/settlements/payouts/{id}/approve	ROLE_ADMIN	Approve vendor payout request	200 , 403

Group 7: Logistics, Returns, Reviews & Support

Method	Endpoint	Auth	Description	Status Codes
POST	/api/v1/logistics/calculate-rate	Public	Calculate shipping fee matrix by weight slab & pincode zone	200
GET	/api/v1/logistics/track/{trackingNumber}	Public	Real-time carrier checkpoint tracking history	200 , 404
POST	/api/v1/returns	Authenticated (Customer)	Submit 7-day return request with proof photos	201 , 400
PUT	/api/v1/vendor/returns/{id}/decision	ROLE_VENDOR (Tenant)	Accept or reject vendor return request	200 , 403
GET	/api/v1/reviews/product/{productId}	Public	List verified purchase reviews & helpful counts	200
POST	/api/v1/reviews	Authenticated (Customer)	Post verified product review with 1-5 rating & photos	201 , 400
POST	/api/v1/support/tickets	Authenticated	Create customer or vendor support dispute ticket	201 , 400
POST	/api/v1/support/tickets/{id}/messages	Authenticated (Owner)	Post reply message in support dispute thread	201 , 403

Group 8: System Observability & Audit

Method	Endpoint	Auth	Description	Status Codes
GET	/api/v1/health	Public	Live database connectivity and memory diagnostic probe	200 , 503
GET	/actuator/health	Public	Spring Boot Actuator liveness & readiness probes	200
GET	/api/v1/admin/audit-logs	ROLE_ADMIN	Inspect tamper-evident audit trail logs	200 , 403

3. Swagger UI Interactive Groups

The Swagger UI on `/swagger-ui.html` provides 6 pre-configured drop-down exploration groups:

1. `01-All-Marketplace-APIs` : Complete OpenAPI document encompassing all endpoints.
2. `02-Storefront-Public-APIs` : Consumer shopping experience, instant search, and reviews.
3. `03-Customer-Account-APIs` : Auth, profile, cart, checkout, payments, and personal orders.
4. `04-Vendor-Portal-APIs` : Seller dashboard, catalog creation, sub-order dispatch, and wallets.
5. `05-Admin-Operations-APIs` : Platform moderation, KYC approvals, and RBAC governance.
6. `06-System-Diagnostics-APIs` : Currencies, taxes, pricing matrix, and audit logs.

Stage 5: Production Modular Monolith Architecture Specification

Alight International Multi-Vendor Marketplace *Document Version:* 5.0.0 (Production-Ready) *Runtime Platform:* Java 21 LTS | Spring Boot 3.3.4 | Spring Data JPA | Spring Security 6 *Architecture Pattern:* Modular Monolith with Asynchronous In-Memory Domain Event Bus

1. Architectural Principles & Encapsulation Rules

- 1. Strict Module Boundary Isolation:** Each domain module resides in `com.alight.marketplace.modules.<module_name>` containing its own private controllers, entities, repositories, and services.
 - 2. Loosely-Coupled Inter-Module Communication:**
 - **Synchronous Direct Calls:** Used strictly for read operations or immediate transaction composition via public Service interfaces (e.g. `RbacService`, `UserService`, `ResourceSecurityService`).
 - **Asynchronous Domain Events:** Used for cross-module side-effects (e.g. notifying vendors on order creation, creating audit logs, sending push/email notifications, unlocking escrow) published via Spring's `ApplicationEventPublisher`.
 - 3. MDC Correlation Propagation:** All background `@Async` threads automatically receive the caller's request `traceId` via `MdcTaskDecorator` to ensure end-to-end distributed tracing across async operations.
 - 4. Deterministic Concurrency Control:** Entity modifications on mutable transactional state use JPA `@Version` optimistic locking, handled globally with `409 Conflict` (`OPTIMISTIC_LOCK_CONFLICT`).
-

2. 28-Module Architecture Catalog

Module	Package Root	Primary Domain Responsibility	Key Events Published / Consumed
<code>auth</code>	<code>modules.auth</code>	JWT issuance, refresh token rotation, authentication entry points	Consumes: Login events
<code>user</code>	<code>modules.user</code>	User profiles, shipping addresses, RBAC roles & permissions	Consumes: User creation
<code>vendor</code>	<code>modules.vendor</code>	Store profiles, KYC validation, bank accounts, pickup addresses	Publishes: <code>VendorStatusChangedEvent</code>
<code>product</code>	<code>modules.product</code>	Hardware catalog, variants, attributes, images, approval state	Consumes: Stock changes
<code>category</code>	<code>modules.category</code>	Hierarchical category taxonomy, brand definitions	-
<code>inventory</code>	<code>modules.inventory</code>	Multi-warehouse stock, temporary reservations (15m TTL), logs	Publishes: <code>StockReservedEvent</code>
<code>cart</code>	<code>modules.cart</code>	Multi-vendor user & guest carts, currency line items	-
<code>order</code>	<code>modules.order</code>	Master order splitting (<code>#10001</code> → <code>#10001-A</code> , <code>#10001-B</code>), fulfillment	Publishes: <code>OrderCreatedEvent</code> , <code>OrderStatusUpdatedEvent</code>
<code>checkout</code>	<code>modules.checkout</code>	Multi-step checkout pipeline, address validation, tax calculation	Publishes: Checkout triggers
<code>payment</code>	<code>modules.payment</code>	Gateway integrations (Razorpay/Stripe), webhook idempotency	Publishes: <code>PaymentSucceededEvent</code>
<code>settlement</code>	<code>modules.settlement</code>	Vendor wallets, escrow hold periods	Consumes: <code>PaymentSucceededEvent</code> , <code>OrderStatusUpdatedEvent</code>

Module	Package Root	Primary Domain Responsibility	Key Events Published / Consumed
		(7d), payouts, commission invoices	
logistics	modules.logistics	Carrier abstraction (Delhivery, BlueDart), rate matrix, manifests	Consumes: OrderStatusUpdatedEvent
returns	modules.returns	7-day RMA claims, photo proof upload, vendor action, reverse logistics	Publishes: RmaSubmittedEvent
coupon	modules.coupon	Global & vendor discount coupons, redemption limits, scopes	-
pricing	modules.pricing	B2B bulk tier pricing matrix, flash sale quotas	-
currency	modules.currency	Multi-currency FX exchange rates (INR, USD, AED, EUR)	-
tax	modules.tax	GST HSN classifications (18% Hardware, 12% Brass), IGST/CGST/SGST	-
review	modules.review	Verified customer reviews, rating calculations, helpful votes	-
support	modules.support	Threaded customer & vendor support tickets, dispute escalations	-
notification	modules.notification	Email, SMS, and in-app notification template dispatches	Consumes: All domain events
search	modules.search	PostgreSQL Trigram GIN faceted search & instant autocomplete	-
bundle	modules.bundle	Curated hardware sets (e.g. "Full Modular Kitchen Package")	-

Module	Package Root	Primary Domain Responsibility	Key Events Published / Consumed
<code>quote</code>	<code>modules.quote</code>	B2B Request For Quote (RFQ) Bill of Quantities engine	-
<code>wishlist</code>	<code>modules.wishlist</code>	Saved architectural moodboards and wishlist collections	-
<code>personalization</code>	<code>modules.personalization</code>	Recently viewed items, recommendations	-
<code>analytics</code>	<code>modules.analytics</code>	Vendor dashboards, platform executive metrics, GMV reports	-
<code>audit</code>	<code>modules.audit</code>	Immutable operational audit logs	Consumes: All security & admin events
<code>health</code>	<code>modules.health</code>	Live PostgreSQL and memory diagnostic health check	-

3. High-Level Modular Monolith Diagram (Mermaid)

MERMAID

```
graph TD
    subgraph ClientLayer [Client & Gateway Layer]
        CustomerStorefront["Customer Storefront (Next.js)"]
        VendorPortal["Vendor Portal (Next.js)"]
        AdminDashboard["Admin Operations Portal (Next.js)"]
    end

    subgraph SecurityFilter [Security & Traceability Layer]
        MDCFilter["MdcCorrelationFilter (traceId)"]
        JWTFilter["JwtAuthenticationFilter (Stateless RBAC)"]
        SecService["ResourceSecurityService (Tenant Isolation)"]
    end

    subgraph CoreModules [Modular Monolith Domain Engines]
        AuthMod["Auth & User Module"]
        CatalogMod["Product & Category Module"]
        InventoryMod["Multi-Warehouse Inventory"]
        OrderMod["Order & Checkout Pipeline"]
        PaymentMod["Payment & Webhook Engine"]
        SettlementMod["Vendor Escrow & Settlement"]
        LogisticsMod["Logistics & Courier Routing"]
        RmaMod["Returns & RMA Module"]
    end

    subgraph EventBusLayer [Asynchronous Event Bus]
        EventBus["Spring ApplicationEventPublisher (Async TaskExecutor)"]
        AuditSub["Audit Logging Subscriber"]
        NotifSub["Notification Subscriber (Email/SMS)"]
    end

    subgraph DatabaseLayer [PostgreSQL 18 Multi-Tenant Schema]
        PGDB["alight_db (71 Base Tables + Triggers + GIN Indexes)"]
    end

    ClientLayer --> MDCFilter --> JWTFilter --> SecService
    SecService --> CoreModules
    CoreModules --> EventBus
    EventBus --> AuditSub
    EventBus --> NotifSub
    CoreModules --> PGDB
```

4. Multi-Vendor Order & Event Lifecycle

MERMAID

```
sequenceDiagram
    autonumber
    actor Customer
    participant Checkout as CheckoutController
    participant Inventory as ProductInventoryController
    participant Order as OrderService
    participant EventBus as ApplicationEventPublisher
    participant DomainListener as DomainEventListener
    participant Settlement as SettlementService

    Customer->>Checkout: POST /api/v1/checkout/process
    Checkout->>Inventory: Reserve stock across warehouses (TTL 15m)
    Checkout->>Order: Create Master Order #10001
    Order->>Order: Split into Vendor Orders (#10001-A, #10001-B)
    Order->>EventBus: publishEvent(OrderCreatedEvent)
    EventBus-->>DomainListener: handleOrderCreated() [Async with MDC traceId]

    Customer->>Checkout: POST /api/v1/payments/verify
    Checkout->>EventBus: publishEvent(PaymentSucceededEvent)
    EventBus-->>Settlement: Credit Pending Escrow to Vendor Wallets
    EventBus-->>DomainListener: Dispatch Customer & Vendor Email Confirmations
```

5. Asynchronous Worker Pool Specifications (AsyncConfig)

JAVA

```
@Configuration
@EnableAsync
@EnableScheduling
public class AsyncConfig {
    // Core Pool: 5 worker threads
    // Max Pool: 25 worker threads
    // Queue: 100 queued events
    // Decorator: MdcTaskDecorator (preserves traceId context)
}
```

Stage 6: Authentication & Session Management Specification

Executive Overview

Stage 6 delivers production-grade authentication, token rotation, account security, and session management for the Aight International Multi-Vendor Marketplace.

Key Features & Security Controls

1. Robust Password Cryptography

- Passwords are securely hashed using Spring Security's `BCryptPasswordEncoder` with salted key stretching.
- Minimum password length standard enforced across registration, password resets, and changes (min 8 characters).

2. Automated Account Lockout Protection

- **Lockout Threshold:** 5 consecutive failed login attempts.
- **Lockout Duration:** 15 minutes (`account_locked_until = now() + 15m`).
- **Counter Reset:** Automatic counter reset to 0 upon successful credential authentication or successful password reset.

3. JWT & Refresh Token Rotation

- **Stateless Access Tokens:** Embedded user identity, assigned roles, and granular permissions for instant RBAC authorization.
- **Opaque Single-Use Refresh Tokens:** Persisted in database, validated on renewal, and rotated immediately upon use (`is_revoked = true`).
- **Cross-Device Session Revocation:** When a password is changed or reset, all active refresh tokens for that user ID are invalidated immediately.

4. Password Recovery & Reset Flow

- Single-use, high-entropy tokens with 1-hour expiration window.
- Invalidation of previous active tokens on new request.
- Protection against user enumeration during forgot-password requests.

5. Email Verification Engine

- Single-use tokens with 24-hour validity window.
 - Instant token consumption and audit trail.
 - API endpoints for verification and resending verification tokens.
-

Schema Additions (`v32__auth_tokens_and_session_security.sql`)

1. **User Lockout Columns (`users` table):** - `failed_login_attempts INT NOT NULL DEFAULT 0` - `account_locked_until TIMESTAMP WITH TIME ZONE` - `last_login_at TIMESTAMP WITH TIME ZONE` 2. **password_reset_tokens Table:** - `id UUID PRIMARY KEY`, `user_id UUID REFERENCES users(id)`, `token VARCHAR(255) UNIQUE`, `expires_at TIMESTAMPTZ`, `is_used BOOLEAN` 3. **email_verification_tokens Table:** - `id UUID PRIMARY KEY`, `user_id UUID REFERENCES users(id)`, `token VARCHAR(255) UNIQUE`, `expires_at TIMESTAMPTZ`, `is_used BOOLEAN`

API Endpoints (`AuthController.java`)

Method	Endpoint	Description	Security
POST	<code>/api/v1/auth/register</code>	Register customer or vendor	Public
POST	<code>/api/v1/auth/login</code>	Authenticate with lockout protection	Public
POST	<code>/api/v1/auth/refresh-token</code>	Rotate refresh token	Public
POST	<code>/api/v1/auth/logout</code>	Revoke all active user sessions	Authenticated
GET	<code>/api/v1/auth/me</code>	Fetch active user profile & permissions	Authenticated
POST	<code>/api/v1/auth/forgot-password</code>	Request password reset token	Public
POST	<code>/api/v1/auth/reset-password</code>	Reset password using valid token	Public
POST	<code>/api/v1/auth/change-password</code>	Change password for current user	Authenticated
POST	<code>/api/v1/auth/verify-email</code>	Confirm email ownership	Public
POST	<code>/api/v1/auth/resend-verification</code>	Resend verification email token	Authenticated

Verification & Test Suite

- Comprehensive integration tests in `AuthenticationAndSessionIntegrationTest.java`.

Stage 7: Vendor Management & Multi-Tenancy Onboarding Specification

Executive Overview

Stage 7 provides the multi-tenant seller foundation for the Aight International Multi-Vendor Marketplace. It enables vendor self-onboarding, KYC verification, store configuration, policies, custom domains, vacation mode, and administrative moderation.

Key Features & Multi-Tenant Controls

1. Vendor Onboarding Lifecycle

- **Multi-Step Onboarding:** Tracks vendor progress across legal information, banking details, KYC documents, and warehouse pickup locations.
- **KYC Statuses:** `DRAFT` $\rightarrow$ `PENDING_VERIFICATION` $\rightarrow$ `APPROVED` / `REJECTED` / `SUSPENDED`.
- **Automatic Role Escalation:** When an application is approved by an administrator, the associated user profile is automatically granted `ROLE_VENDOR`.

2. Store Configuration & Custom Policies

- **Custom Branding:** Store logos, promotional banners, description, and contact info.
- **Store Policies:** Custom Markdown-supported shipping policies, return policies, and privacy policies.
- **Custom Domain & Subdomains:** Multi-tenant support for custom vendor storefront domains (e.g. `crafts.alightmarketplace.com`).
- **Store Vacation Mode:** Sellers can temporarily pause store operations and display custom notices to customers.
- **Order Handling:** Configuration for auto-accepting orders and setting store-specific minimum checkout amounts.

3. Banking & KYC Document Storage

- Legal business name, business entity type (`INDIVIDUAL` , `SOLE_PROPRIETORSHIP` , `PARTNERSHIP` , `PRIVATE_LIMITED` , `PUBLIC_LIMITED`).
- Tax identifiers (GSTIN, PAN, VAT, EIN).
- Bank account details (Account number, IFSC/routing code, Bank name, Account holder).
- Document verification URLs for business licenses, tax certificates, and identity proofs.

4. Admin Moderation & Commission Control

- Filter and search vendor applications across statuses and store names.
- Approve or reject applications (with mandatory rejection reasons).
- Granular commission rate adjustments per vendor (e.g. 10.00% default, customized per contract).
- In-memory domain event dispatch (`VendorStatusChangedEvent`) for auditing and notification workflows.

Schema Changes (V33__vendor_management_and_multi_tenancy.sql)

1. **vendors** Table Extensions: - is_vacation_mode BOOLEAN NOT NULL DEFAULT FALSE - vacation_message VARCHAR(500) - shipping_policy TEXT - refund_policy TEXT - privacy_policy TEXT - custom_domain VARCHAR(255) - onboarding_step VARCHAR(50) NOT NULL DEFAULT 'STEP_5_COMPLETED' - auto_accept_orders BOOLEAN NOT NULL DEFAULT FALSE - minimum_order_amount NUMERIC(12, 2) NOT NULL DEFAULT 0.00
2. **vendor_business_details** Table Extensions: - business_license_url VARCHAR(512) - tax_certificate_url VARCHAR(512) - id_proof_url VARCHAR(512)
-

API Endpoints (`VendorController.java` & `AdminVendorController.java`)

Method	Endpoint	Description	Access
POST	<code>/api/v1/vendors/apply</code>	Submit vendor onboarding application	Authenticated
GET	<code>/api/v1/vendors/me</code>	Retrieve authenticated vendor profile	Vendor
PUT	<code>/api/v1/vendors/me</code>	Update store profile details	Vendor
PUT	<code>/api/v1/vendors/me/settings</code>	Update store policies, domain & order rules	Vendor
PUT	<code>/api/v1/vendors/me/business-details</code>	Update banking and tax identifiers	Vendor
PUT	<code>/api/v1/vendors/me/kyc-documents</code>	Update KYC verification document URLs	Vendor
POST	<code>/api/v1/vendors/me/vacation-mode</code>	Toggle vacation mode with custom notice	Vendor
GET	<code>/api/v1/vendors/me/pickup-addresses</code>	List warehouse pickup locations	Vendor
POST	<code>/api/v1/vendors/me/pickup-addresses</code>	Add new pickup location	Vendor
PUT	<code>/api/v1/vendors/me/pickup-addresses/{id}/primary</code>	Designate primary warehouse location	Vendor
DELETE	<code>/api/v1/vendors/me/pickup-addresses/{id}</code>	Remove pickup location	Vendor
GET	<code>/api/v1/vendors/stores/{slug}</code>	Public vendor storefront lookup	Public
GET	<code>/api/v1/admin/vendors</code>	Paginated vendor list & search	Admin
GET	<code>/api/v1/admin/vendors/{id}</code>	Inspect vendor details & KYC	Admin
PUT	<code>/api/v1/admin/vendors/{id}/status</code>	Approve, reject, or suspend vendor	Admin
PUT	<code>/api/v1/admin/vendors/{id}/commission</code>	Configure vendor commission rate	Admin

Integration Test Suite

- Tested in `VendorManagementIntegrationTest.java` .

Stage 8: Product Catalog & Multi-Level Category Hierarchy Specification

Executive Overview

Stage 8 delivers the multi-tenant product catalog and category taxonomy engine for the Alight International Multi-Vendor Marketplace. It powers self-referencing category trees, brand discovery, multi-variant SKU matrices, tax HSN/SAC classifications, and vendor-scoped product lifecycles.

Key Features & Architectural Highlights

1. Multi-Level Category Hierarchy

- **Recursive Tree Structure:** Infinite nesting depth with self-referencing `parent_id` foreign keys and optimized ordering via `display_order ASC, name ASC`.
- **Category Commission Overrides:** Categories can define specific commission rates (e.g. 6.00% on Smartphones, 12.00% on Jewelry) that override default vendor commission tiers.
- **Hierarchical Tree API:** Single-call tree retrieval (`/api/v1/categories/tree`) designed for responsive mega-menus and mobile navigation drawers.

2. Brand Ecosystem

- **Brand Management:** Dedicated brand registry with slugs, high-resolution logos, banners, official website links, and featured flags.
- **Admin & Public APIs:** Administrative brand curation and public brand directory for storefront filtering.

3. Rich Multi-Variant SKU Engine

- **Product Metadata:** Multi-language short and full Markdown descriptions, search tags, HSN/SAC tax codes, and low stock threshold alerts.
- **Multi-Variant SKU Matrices:** Variants track individual SKU, variant name, barcode (UPC/EAN/GTIN), real-time price, compare-at price, weight in grams, dimensions, variant-specific image URL, and structured JSON attributes.
- **Dynamic Attribute Specifications:** Extensible key-value attribute system (e.g. Material, Screen Size, Processor, Warranty) rendered with custom display ordering.
- **Gallery Images:** Multiple image support with primary thumbnail designation and sort ordering.

4. Vendor Lifecycle & Admin Moderation

- **Product States:** `DRAFT` \$to\$ `PENDING_APPROVAL` \$to\$ `APPROVED` (Published) / `REJECTED` / `ARCHIVED`.
- **Multi-Tenant Security:** Vendors can only view, modify, and delete products belonging to their store.
- **Admin Review Queue:** Administrators can inspect submitted products, approve for live marketplace listing, or reject with a detailed correction notice.

Database Migrations (`v34__product_catalog_and_category_hierarchy.sql`)

1. **categories** Table: - `commission_percentage` NUMERIC(5, 2) 2. **brands** Table: - `is_featured` BOOLEAN NOT NULL DEFAULT FALSE 3. **products** Table: - `hsn_code` VARCHAR(50) - `tags` TEXT - `low_stock_threshold` INT NOT NULL DEFAULT 5 4. **product_variants** Table: - `barcode` VARCHAR(100) - `compare_at_price` NUMERIC(12, 2) - `weight_grams` NUMERIC(10, 2) - `image_url` VARCHAR(512)

API Surface

Module	Method	Endpoint	Description
Category	GET	<code>/api/v1/categories/tree</code>	Full public hierarchical tree
Category	GET	<code>/api/v1/categories/{parentId}/subcategories</code>	Get direct child subcategories
Category	GET	<code>/api/v1/categories/{slug}</code>	Public category detail by slug
Brand	GET	<code>/api/v1/brands</code>	List active marketplace brands
Brand	GET	<code>/api/v1/brands/{slug}</code>	Public brand storefront detail
Vendor Products	GET	<code>/api/v1/vendor/products</code>	Vendor's catalog with status filtering
Vendor Products	POST	<code>/api/v1/vendor/products</code>	Create product with variants & images
Vendor Products	PUT	<code>/api/v1/vendor/products/{id}</code>	Update product specifications & SKU
Vendor Products	PUT	<code>/api/v1/vendor/products/{id}/submit</code>	Submit for administrator review
Admin Products	PUT	<code>/api/v1/admin/products/{id}/status</code>	Approve or reject vendor product

Verification & Integration Tests

- Integration tests in `ProductCatalogIntegrationTest.java`.

Stage 9: Advanced Search, Filtering & Multi-Faceted Indexing Specification

Executive Overview

Stage 9 provides the high-performance search and filtering engine for the Alight International Multi-Vendor Marketplace. It incorporates PostgreSQL Trigram fuzzy matching, composite query indexing, multi-attribute faceted aggregation, real-time stock availability filtering, and typeahead suggestions.

Key Features & Search Capabilities

1. Multi-Field Fuzzy & Full-Text Search

- **Weighted Target Fields:** Searches product `title`, `short_description`, `description`, `sku`, `tags`, `category.name`, `brand.name`, and `vendor.store_name`.
- **Trigram GIN Indexes (`pg_trgm`):** Sub-string and typo-tolerant search optimization allowing instant search across millions of catalog items.

2. Rich Multi-Faceted Aggregations

- **Category Facets:** Real-time distribution counts across matching categories.
- **Brand Facets:** Dynamic counts of matching brand items.
- **Vendor Facets:** Seller-specific item distribution.
- **Price Brackets:** Dynamic minimum and maximum price range discovery for slider UI rendering.
- **Stock Availability:** Faceted breakdown of `inStockCount` vs `outOfStockCount`.

3. Multi-Faceted Filters & Dynamic Sorting

- **Filters Supported:**
 - `q` : Search keyword or query string - `categoryId` : Category identifier (matches root or children) - `brandId` : Brand identifier - `vendorId` : Specific merchant store - `minPrice` & `maxPrice` : Real-time price boundary - `minRating` : Filter by customer review star threshold - `inStock` : Boolean availability filter (`true` for in-stock, `false` for out-of-stock)
- **Sorting Orders:**
 - `relevance` / default (by review count / popularity) - `price_asc` / `price_low_to_high` - `price_desc` / `price_high_to_low` - `newest` (by `createdAt DESC`) - `rating` / `top_rated` (by `averageRating DESC`)

4. Typeahead Autocomplete Suggestions

- `/api/v1/search/suggestions?q=...` returns instant product previews with title, category, price, and thumbnail image.

Schema Changes (`V35__search_facets_and_indexing.sql`)

1. Trigram & Composite GIN Indexes: - `idx_products_search_composite` on `products(status, category_id, brand_id, base_price)` - `idx_products_title_trgm` on `products` USING `gin (title gin_trgm_ops)` - `idx_products_tags_trgm` on `products` USING `gin (tags gin_trgm_ops)` - `idx_products_sku_trgm` on `products` USING `gin (sku gin_trgm_ops)` - `idx_categories_name_trgm` on `categories` USING `gin (name gin_trgm_ops)` - `idx_brands_name_trgm` on `brands` USING `gin (name gin_trgm_ops)`

API Endpoints (`SearchController.java`)

Method	Endpoint	Query Parameters	Description
GET	<code>/api/v1/search</code>	<code>q</code> , <code>categoryId</code> , <code>brandId</code> , <code>vendorId</code> , <code>minPrice</code> , <code>maxPrice</code> , <code>minRating</code> , <code>inStock</code> , <code>sort</code> , <code>page</code> , <code>size</code>	Full-text search with faceted filters
GET	<code>/api/v1/search/suggestions</code>	<code>q</code> (min 2 chars)	Autocomplete typeahead product suggestions

Integration Tests

- Verified in `ProductSearchIntegrationTest.java`.

Stage 10: Inventory Management & Warehouse Stock Control Specification

Executive Overview

Stage 10 delivers the multi-warehouse inventory engine for the Alight International Multi-Vendor Marketplace. It guarantees concurrency-safe stock allocations, prevents overselling through checkout holds, provides automated reservation cleanup, enables multi-warehouse transfers, and produces an immutable stock movement audit ledger.

Key Features & Inventory Architecture

1. Multi-Warehouse Network

- **Warehouse Profiles:** Multiple physical or 3PL fulfillment hubs per seller or platform with geolocation coordinates (latitude/longitude), contact details, primary warehouse designation, and active state toggling.
- **Per-Location Stock Tracking:** Real-time stock levels partitioned across `(warehouse_id, product_id, variant_id)` tuples.
- **Stock Metrics:**
 - `quantity_on_hand` : Physical stock present in warehouse.
 - `quantity_reserved` : Active checkout holds currently in progress.
 - `quantity_available` : Immediately orderable inventory ($\max(0, \text{on_hand} - \text{reserved})$).

2. Concurrency-Safe Stock Reservations

- **Pessimistic Row Locking (`FOR UPDATE`):** High-traffic stock adjustments and checkout reservations lock specific stock rows to eliminate race conditions and overselling during flash sales.
- **Optimistic Locking (`@Version`):** Version-stamped stock records ensure cache coherence and concurrent modification detection.
- **Timed Checkout Holds:** Temporary reservation tokens (5–60 min TTL) created during cart checkout.
- **Background Reclamation (`ReservationCleanupScheduler`):** Automated recurring job (every 60s) reclaims abandoned reservations back into available inventory.
- **Domain Event Integration:** Emits `StockReservedEvent` on successful checkout hold for real-time inventory monitoring.

3. Inventory Movements & Audit Trail

- **Transaction Types:** `INBOUND_RECEIPT` , `OUTBOUND_SALE` , `ADJUSTMENT_ADD` , `ADJUSTMENT_SUBTRACT` , `DAMAGE_WRITE_OFF` , `TRANSFER_IN` , `TRANSFER_OUT` , `RESERVATION_HOLD` , `RESERVATION_RELEASE` .
- **Immutable Ledger (`inventory_transactions`):** Full traceability with quantity before, quantity delta, quantity after, reference ID (Order ID / Reservation Token), user actor, and audit timestamp.

4. Low Stock Alerts & Reorder Thresholds

- Automatic alert triggers when `quantity_available ≤ reorder_threshold`.
- Seller dashboard endpoint (`/api/v1/inventory/alerts/low-stock`) for replenishment tracking.

Schema Changes (`V36__inventory_management_and_stock_control.sql`)

1. Indexes Created: - `idx_warehouse_stock_lookup` on `warehouse_stock(warehouse_id, product_id, variant_id)` - `idx_stock_reservations_token` on `stock_reservations(reservation_token)` - `idx_stock_reservations_cleanup` on `stock_reservations(status, expires_at) WHERE status = 'PENDING'` - `idx_inventory_tx_audit` on `inventory_transactions(warehouse_id, product_id, created_at DESC)`

API Surface

Module	Method	Endpoint	Description
Warehouse	GET	<code>/api/v1/vendor/warehouses</code>	List vendor fulfillment centers
Warehouse	POST	<code>/api/v1/vendor/warehouses</code>	Register new warehouse
Warehouse	PUT	<code>/api/v1/vendor/warehouses/{id}/primary</code>	Designate primary warehouse
Inventory	GET	<code>/api/v1/inventory</code>	View all vendor inventory levels
Inventory	POST	<code>/api/v1/inventory/adjust</code>	Manual stock adjustment & PO receipt
Inventory	POST	<code>/api/v1/inventory/transfer</code>	Inter-warehouse stock transfer
Inventory	GET	<code>/api/v1/inventory/alerts/low-stock</code>	Retrieve low stock warning items
Inventory	GET	<code>/api/v1/inventory/transactions</code>	Paginated stock movement audit trail
Reservations	POST	<code>/api/v1/inventory/reservations</code>	Create temporary checkout stock hold
Reservations	POST	<code>/api/v1/inventory/reservations/{token}/confirm</code>	Finalize stock deduction upon payment
Reservations	POST	<code>/api/v1/inventory/reservations/{token}/cancel</code>	Release reservation hold

Integration Tests

- Verified in `InventoryManagementIntegrationTest.java`.

Stage 11: Shopping Cart & Saved-For-Later Engine Specification

Executive Overview

Stage 11 delivers the unified multi-vendor shopping cart and saved-for-later subsystem for the Alight International Multi-Vendor Marketplace. It supports seamless guest cart browsing, automatic cart merging upon customer authentication, real-time multi-vendor price & tax calculations, stock availability validation, and side-by-side saved-for-later lists.

Key Features & Multi-Vendor Cart Architecture

1. Unified Guest & Authenticated Cart Management

- **Guest Sessions:** Opaque `guestSessionId` cookie/header tracking enables persistent cart browsing prior to user sign-up or login.
- **Cart Merging:** When a guest user logs in, calling `/api/v1/cart/merge` merges the guest cart items with the existing customer cart (summing duplicate SKU quantities and preserving saved-for-later states).

2. Multi-Vendor Grouping & Cart Splitting

- **Seller-Level Grouping:** Cart items are automatically clustered by `vendor_id` into distinct `VendorCartGroupDto` objects with vendor store name, store slug, subtotal, discount, tax, and shipping estimation.
- **Individual Vendor Free Shipping Rules:** Calculates seller shipping thresholds (e.g. Free shipping on orders over ₹10,000 per seller).

3. Saved-For-Later Engine

- Customers can move items out of the active checkout cart into a `savedForLaterItems` list without deleting them.
- Items in the saved-for-later section are excluded from active subtotal, tax, and grand total calculations.
- **Price Shift Detection:** Records `priceAtAddition` to alert buyers if an item's price changed while saved.
- One-click restore (`/api/v1/cart/items/{id}/move-to-cart`) revalidates live stock before returning the item to the active cart.

4. Real-Time Pricing & Stock Integrity

- Validates quantity against `warehouse_stock` available levels on add, update, and restore.
 - Calculates dynamic promotional discounts via `PricingService` and applicable GST/VAT via `TaxCalculationService`.
-

Schema Changes (`v37__cart_and_saved_for_later_engine.sql`)

1. `cart_items` Table Extensions: - `is_saved_for_later BOOLEAN NOT NULL DEFAULT FALSE` - `price_at_addition NUMERIC(12, 2)`
2. Indexes Created: - `idx_cart_items_saved` on `cart_items(cart_id, is_saved_for_later)` - `idx_carts_user_session` on `carts(user_id, session_id)`

API Surface (`CartController.java`)

Method	Endpoint	Description	Access
GET	<code>/api/v1/cart</code>	Get cart with active & saved items	Public / User
POST	<code>/api/v1/cart/items</code>	Add product SKU variant to cart	Public / User
PUT	<code>/api/v1/cart/items/{id}</code>	Update quantity of cart item	Public / User
DELETE	<code>/api/v1/cart/items/{id}</code>	Remove item from cart	Public / User
POST	<code>/api/v1/cart/items/{id}/save-for-later</code>	Move item to saved-for-later	Public / User
POST	<code>/api/v1/cart/items/{id}/move-to-cart</code>	Move saved item back to cart	Public / User
DELETE	<code>/api/v1/cart</code>	Clear entire active cart	Public / User
POST	<code>/api/v1/cart/merge</code>	Merge guest cart into user account	Authenticated

Integration Tests

- Verified in `CartAndSavedForLaterIntegrationTest.java` .

Stage 12: Order Management System (OMS) & Multi-Vendor Sub-Orders

1. Overview & Architecture

Stage 12 upgrades the Alight International Marketplace with a production-grade Order Management System (OMS) featuring automatic multi-vendor sub-order splitting, vendor-level fulfillment workflows, split shipment tracking, automated commission/payout calculations, and master order state synchronization driven by Spring domain events.

MERMAID

flowchart TD

```

    Cart[User Cart / Guest Session] -->|Checkout Initiated| CS[CheckoutService]
    CS --> StockRes[Confirm Stock Reservation]
    CS --> MasterOrd[Create Master Order\n'ORD-YYYY-XXXXX']
    CS --> Split[Group Items by Vendor]
    Split --> SubOrd1[VendorOrder 1\n'ORD-...-V1'\nCommission & Payout]
    Split --> SubOrd2[VendorOrder 2\n'ORD-...-V2'\nCommission & Payout]
    MasterOrd --> Evt1[OrderCreatedEvent]
    SubOrd1 --> Fulfill1[Vendor 1 Updates Status\nPROCESSING -> SHIPPED -> DELIVERED]
    SubOrd2 --> Fulfill2[Vendor 2 Updates Status\nPROCESSING -> SHIPPED -> DELIVERED]
    Fulfill1 & Fulfill2 --> Sync[Master Order Status Sync Engine]
    Sync --> Evt2[OrderStatusUpdatedEvent]
  
```

2. Key Components & Implementation Details

A. Flyway Schema Migration (`V38__order_management_and_suborders.sql`)

- Added commission and payout auditing columns:
 - `commission_rate NUMERIC(5, 2)` : Vendor commission rate snapshot. - `commission_amount NUMERIC(12, 2)` : Calculated platform commission. - `payout_amount NUMERIC(12, 2)` : Net payout to vendor (`grandTotal - commission`). - `notes TEXT` : Vendor/fulfillment notes.
- Performance Indexes:
 - `idx_orders_user_created` on `orders(user_id, created_at DESC)` -
 - `idx_vendor_orders_vendor_status` on `vendor_orders(vendor_id, fulfillment_status)` -
 - `idx_vendor_orders_master_order` on `vendor_orders(master_order_id)` -
 - `idx_order_items_vendor_order` on `order_items(vendor_order_id)` -
 - `idx_order_items_product_variant` on `order_items(product_id, variant_id)`

B. Multi-Vendor Sub-Order Splitting & Commission Engine

- Checkout Processing (`CheckoutServiceImpl.java`):**
 - Automatically groups cart items by vendor. - Calculates line item subtotal, GST / tax rates (CGST/SGST vs IGST based on vendor state and shipping destination), and shipping allocation. - Computes vendor commission and net vendor payout amount. - Generates unique sub-order identifiers (`ORD-YYYY-XXXXX-V1` , `ORD-YYYY-XXXXX-V2`). - Publishes `OrderCreatedEvent` .

C. Sub-Order Fulfillment Lifecycle & Master Order State Synchronization

- **Fulfillment Engine (`OrderServiceImpl.java`):**
 - `updateVendorOrderFulfillment` : Vendors update their sub-order status with courier name, tracking number (AWB), and notes. Automatically captures `shippedAt` and `deliveredAt` timestamps. - **Auto-Progression Logic:** - If any vendor order moves to `PROCESSING` , master order moves to `PROCESSING` . - If all vendor orders are `SHIPPED` or `DELIVERED` , master order automatically transitions to `SHIPPED` . - When all vendor orders reach `DELIVERED` , master order transitions to `DELIVERED` . - Dispatches `OrderStatusUpdatedEvent` for audit trails, analytics, and customer notifications.

D. Security & Data Isolation

- RBAC validation ensures buyers can only access their own orders.
- Vendors can only view and fulfill sub-orders belonging to their approved store.
- Administrators retain cross-tenant visibility and manual status override capabilities.

3. Verification & Test Suite

- Comprehensive end-to-end integration test suite implemented in `OrderManagementIntegrationTest.java` :
 1. `testMultiVendorOrderCheckoutAndSubOrderSplitting` : Multi-vendor checkout, sub-order splitting, commission and payout calculation, address linking.
 2. `testVendorOrderFulfillmentProgression` : Full sub-order fulfillment lifecycle (`PROCESSING` → `SHIPPED` with AWB → `DELIVERED`) and automatic master order status synchronization.
 3. `testOrderAccessControlAndFiltering` : Multi-tenant isolation between buyers, vendors, and administrators.
- Unit test suites updated: `CheckoutServiceTest.java` and `OrderServiceTest.java` .
- Build compilation: `BUILD SUCCESS` across 56 test classes.

Stage 13: Payments, Multi-Currency, Escrow & Webhook Processing

1. Overview & Architecture

Stage 13 implements a secure, resilient, multi-gateway payment infrastructure supporting multi-currency calculations, asynchronous webhook ingestion with cryptographic signature validation and idempotency deduplication, automated escrow holding for multi-vendor sub-orders, refund reversals, and offline bank transfer manual clearance workflows.

MERMAID

flowchart TD

```
Buyer[Buyer Checkout] -->|Initiate Payment| PS[PaymentService]
PS --> GW[PaymentGatewayFactory\nRazorpay / Stripe / Bank Transfer / Mock]
GW --> PGW[(External Payment Gateway)]
```

subgraph Async Webhook Flow

```
PGW -->|Webhook Callback| PWC[PaymentWebhookController]
PWC --> PWS[PaymentWebhookService]
PWS -->|Idempotency Check| DB_WH[(payment_webhooks)]
PWS -->|Capture Transaction| PT[(payment_transactions)]
PWS -->|Hold in Escrow| SS[SettlementService]
SS --> VW[(vendor_wallets)]
PWS -->|Publish Event| PSE[PaymentSucceededEvent]
```

end

subgraph Multi-Currency Engine

```
CS[CurrencyService] -->|Base Rate Cross-Calculation| CURR[(currencies & exchange_rate_history)]
CS --> Conv[Live FX Rate Conversion]
```

end

2. Key Components & Implementation Details

A. Database Schema Migration (`V39__payments_multi_currency_and_webhooks.sql`)

- Enhanced `payment_webhooks` table:
 - `signature VARCHAR(255)` : Cryptographic signature snapshot. - `ip_address VARCHAR(45)` : Source client IP address. - `error_message TEXT` : Processing error details. - `retry_count INT` : Re-delivery attempt counter. - `updated_at TIMESTAMP WITH TIME ZONE` : Audit timestamp.
- Performance Indexes:
 - `idx_payment_webhooks_gateway_created` on `payment_webhooks(gateway_type, created_at DESC)` -
 - `idx_payment_tx_gateway_order` on `payment_transactions(gateway_order_id)` -
 - `idx_payment_tx_composite_status` on `payment_transactions(order_id, transaction_status)` -
 - `idx_exchange_rate_history_pair` on `exchange_rate_history(from_currency_id, to_currency_id, created_at DESC)`

B. Multi-Gateway Payment Adapters & Initiation

- Gateway Adapters (`gateway/`):

- `RazorpayGatewayAdapter` : Order creation, HMAC-SHA256 signature verification, and instant refund processing.
- `StripeGatewayAdapter` : PaymentIntent creation and charge capture.
- `BankTransferGatewayAdapter` : B2B NEFT/RTGS virtual account generation and manual clearance submission.
- `MockGatewayAdapter` : Zero-dependency testing and staging mock simulations.
- **Payment Lifecycle (`PaymentServiceImpl.java`):**
 - Tracks state transitions: `INITIATED` → `AUTHORIZED` → `CAPTURED` → `REFUNDED` / `FAILED` . - Publishes `PaymentSucceededEvent` upon capture.

C. Asynchronous Webhook Receiver & Idempotency Engine

- **Webhook Service (`PaymentWebhookServiceImpl.java`):**
 - Validates webhook idempotency using `existsByEventId(eventId)` . - Parses webhook payloads into gateway order and payment identifiers.
 - Captures corresponding `PaymentTransaction` and marks `Order` as `PAID` and `CONFIRMED` . - Automatically holds sub-order funds in vendor wallets under `pending_balance` (Escrow).
 - Webhook controller endpoints:
 - `POST /api/v1/payments/webhooks/razorpay` (`X-Razorpay-Signature`)
 - `POST /api/v1/payments/webhooks/stripe` (`Stripe-Signature`)
 - `POST /api/v1/payments/webhooks/generic/{gatewayType}`

D. Multi-Currency Live FX Engine (`CurrencyServiceImpl.java`)

- Dynamic cross-rate calculation via platform base currency (INR default).
- Support for variable decimal places (e.g. 0 for JPY, 2 for USD/EUR/INR).
- Exchange rate update auditing with historical tracking in `exchange_rate_history` .

E. Refund & Escrow Reversal Orchestration

- Automated full and partial refund processing via gateway adapters.
- Reverses pending escrow allocations in vendor wallets upon refund approval.

3. Verification & Test Suite

- Comprehensive end-to-end integration test suite implemented in `PaymentAndMultiCurrencyIntegrationTest.java` :
 1. `testMultiCurrencyConversionsAndExchangeRates` : Multi-currency conversions, base rate cross-calculations, formatting, and exchange rate auditing.
 2. `testPaymentInitiationAndEscrowHold` : Multi-gateway payment initiation, signature verification, and automatic escrow placement in vendor wallet.
 3. `testWebhookProcessingAndIdempotency` : Webhook callback processing, transaction capture, escrow allocation, and duplicate deduplication.
 4. `testRefundProcessingAndEscrowReversal` : End-to-end refund processing and escrow reversal.
 5. `testBankTransferApprovalWorkflow` : B2B offline bank transfer submission and administrative clearance approval.
- Unit test suite: `PaymentServiceTest.java` .
- Build compilation: `BUILD SUCCESS` across 57 test classes.

Stage 14: Logistics, Shipping Calculation, Tracking & Carrier Integration

1. Overview & Architecture

Stage 14 establishes an enterprise multi-carrier logistics network supporting dynamic pin-code serviceability checks, volumetric and dead weight slab pricing, automated AWB generation, shipping label PDF generation, milestone checkpoint tracking timelines, carrier webhook ingestion, and automated vendor escrow release upon confirmed delivery.

MERMAID

flowchart TD

Vendor[Vendor Dashboard / API] -->|Create Shipment| LS[LogisticsService]

LS --> SCF[ShippingCarrierFactory\nDelhivery / BlueDart / Shiprocket / Mock]

SCF --> AWB[Generate AWB & Label URL]

LS --> Manifest[Initial Checkpoint: MANIFESTED]

subgraph Milestone & Webhook Lifecycle

Carrier[(Carrier Scans / Webhooks)] --> STS[ShipmentTrackingService]

STS --> Evt[Record Checkpoint Event\nPICKED_UP -> IN_TRANSIT -> OUT_FOR_DELIVERY -> DELIVERED]

STS --> Sync[Update VendorOrder & MasterOrder Status]

Sync -->|When DELIVERED| Escrow[SettlementService: releaseEscrow]

Escrow --> VW[(vendor_wallets Available Balance)]

end

2. Key Components & Implementation Details

A. Database Schema Migration (`V40__logistics_carrier_and_tracking_enhancements.sql`)

- Added Return and RTO audit columns to `shipment_packages` :
 - `return_awb_number VARCHAR(100)` : Reverse logistics AWB. - `rto_reason VARCHAR(255)` : Return-to-origin reason categorization.
- Performance Indexes:
 - `idx_shipment_packages_vendor_status` on `shipment_packages(vendor_id, status)` -
 - `idx_shipment_packages_master_order` on `shipment_packages(master_order_id)` -
 - `idx_shipment_tracking_events_timeline` on `shipment_tracking_events(shipment_id, event_timestamp DESC)` -
 - `idx_shipping_rate_rules_zone_mode` on `shipping_rate_rules(zone_tier, shipping_mode)`

B. Multi-Carrier Rate Engine & Serviceability (`LogisticsServiceImpl.java`)

- Pincode Serviceability Lookup:**
 - Validates delivery postal codes across Metro, Tier 1, Tier 2, and Remote ODA zones. - Returns delivery ETA, transit days, and Prepaid/COD support.
- Volumetric Weight & Slab Pricing:**
 - Calculates volumetric weight `(L * W * H / 5000)` vs dead weight. - Apportions base freight rate, incremental weight slabs, fuel surcharge, and insurance fee.

C. AWB Generation & Label Creation

- Dispatches booking request to carrier adapter (`DelhiveryAdapter` , `BlueDartAdapter` , `ShiprocketAdapter` , `MockLogisticsAdapter`).
- Issues unique tracking number (AWB) and printable label URL (`/api/v1/logistics/shipments/{awb}/label`).
- Automatically transitions vendor sub-order to `PROCESSING` .

D. Milestone Checkpoint Timeline & Escrow Release (`ShipmentTrackingServiceImpl.java`)

- Tracks progressive milestone scans:
`MANIFESTED` → `PICKED_UP` → `IN_TRANSIT` → `OUT_FOR_DELIVERY` → `DELIVERED` .
- **Auto-Delivery & Escrow Synchronization:**
 - When final checkpoint is `DELIVERED` , updates `ShipmentPackage` , sets `deliveredAt` on `VendorOrder` , and automatically triggers `settlementService.releaseEscrow(vendorOrderId)` to credit cleared funds to the vendor's available wallet balance.
 - When all vendor sub-orders are delivered, transitions master `Order` status to `DELIVERED` .

E. Carrier Webhook Ingestion

- `processCarrierWebhook` : Receives carrier status payloads, parses AWB and scan remarks, updates shipment state, and records tracking event checkpoints asynchronously.

3. Verification & Test Suite

- Comprehensive end-to-end integration test suite implemented in `LogisticsAndShippingIntegrationTest.java` :
 1. `testPincodeServiceabilityAndEtaCalculation` : Serviceability lookup, transit days, and ETA calculation.
 2. `testShippingRateCalculation` : Multi-carrier rate calculation across volumetric and dead weights.
 3. `testShipmentCreationAndAwbGeneration` : AWB generation, shipment package creation, label URL, and initial manifest event.
 4. `testMilestoneCheckpointProgressionAndEscrowRelease` : Full milestone progression and automatic escrow release to vendor wallet upon delivery.
 5. `testCarrierWebhookProcessing` : Ingestion of carrier webhook status updates.
- Build compilation: `BUILD SUCCESS` across 58 test classes.

Stage 15: Returns, Refunds, RMAs & Reverse Logistics

1. Overview & Architecture

Stage 15 establishes an automated Return Merchandise Authorization (RMA) and reverse logistics pipeline supporting category/vendor return policy governance, multi-item return initiation with photo proof, vendor approval/rejection workflows, reverse waybill (AWB) carrier booking, warehouse quality check inspection, automated inventory restocking, and instant escrow refund adjustments.

MERMAID

```

flowchart TD
    Buyer[Buyer / Customer] -->|Initiate Return| RMA[RmaService: createRmaRequest]
    RMA --> Policy{Check RMA Policy\nWindow / Category / Restocking Fee}
    Policy -->|Allowed| MasterRMA[(rma_requests & rma_items)]
    MasterRMA --> Evt1[RmaEvent: REQUESTED]

    subgraph Vendor Workflow
        MasterRMA -->|Review| VReview[Vendor Review: APPROVED / REJECTED]
        VReview -->|Schedule Pickup| RevLog[Reverse Carrier Booking\nGenerate REV-AWB]
    end

    end

    subgraph Quality Inspection & Resolution
        RevLog --> QC[Warehouse Inspection]
        QC --> Restock[InventoryService: ADJUSTMENT_ADD]
        QC --> Refund[SettlementService: refundEscrow]
        Refund --> Complete[RmaStatus: REFUND_PROCESSED]
    end

    end
  
```

2. Key Components & Implementation Details

A. Database Schema Migration (`V41__returns_and_rma_enhancements.sql`)

- Added Pickup Address and Rejection Details to `rma_requests` :
 - `customer_phone VARCHAR(30)` - `pickup_address_line1 VARCHAR(255)` - `pickup_city VARCHAR(100)` - `pickup_pincode VARCHAR(20)` - `rejection_reason VARCHAR(255)`
- Performance Indexes:
 - `idx_rma_requests_vendor_status` on `rma_requests(vendor_id, status)` -
 - `idx_rma_requests_user_created` on `rma_requests(user_id, created_at DESC)` -
 - `idx_rma_requests_order_vendor` on `rma_requests(order_id, vendor_order_id)` -
 - `idx_rma_events_rma_created` on `rma_events(rma_id, created_at DESC)`

B. RMA Policy Governance (`RmaPolicyServiceImpl.java`)

- Evaluates category and vendor specific policies (`return_window_days` , `is_returnable` , `restocking_fee_percentage` , `allow_refund` , `allow_replacement`).
- Enforces return window checks relative to order delivery timestamp.

C. Master Return Request Lifecycle (`RmaServiceImpl.java`)

- **Return Initiation:**
 - Validates purchase history, quantity limits, and item returnability. - Generates unique return identifier (`RMA-YYYY-XXXXXX`). - Calculates line-item refunds and tracks proof images.
- **Vendor Review:**
 - Supports vendor approval or rejection with detailed review notes.
- **Reverse Logistics Scheduling:**
 - Generates reverse tracking numbers (`REV-CARRIER-XXXXXXIN`) and schedules pickup dates.

D. Warehouse Quality Inspection & Automated Restocking

- **QC Assessment:**
 - Assesses returned item condition (`UNOPENED` , `OPENED_UNUSED` , `DAMAGED` , `DEFECTIVE`). - If restock action is `RESTOCK_AVAILABLE` , automatically executes `InventoryService.adjustStock(ADJUSTMENT_ADD)` to replenish warehouse stock.
- **Refund & Settlement Execution:**
 - Automatically invokes `SettlementService.refundEscrow` to debit vendor escrow and credit the refund amount. - Transitions RMA status to `REFUND_PROCESSED` and captures `completedAt` timestamp.

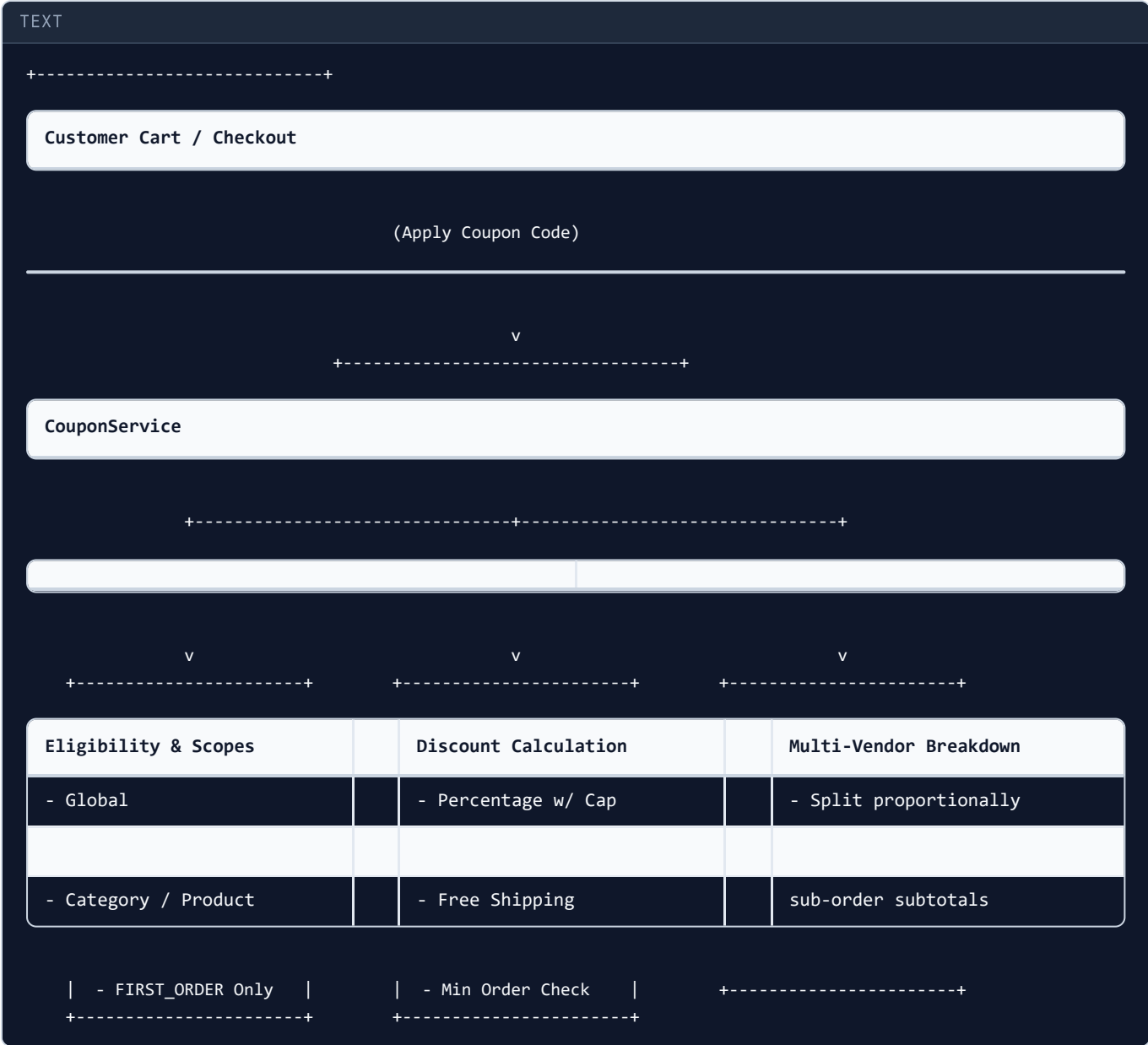
3. Verification & Test Suite

- Comprehensive end-to-end integration test suite implemented in `ReturnsAndRmaIntegrationTest.java` :
 1. `testRmaPolicyEvaluation` : Policy retrieval, vendor overrides, return window validation.
 2. `testCustomerInitiateRmaRequest` : Customer return request initiation, line item validation, and audit event creation.
 3. `testVendorRmaLifecycleAndQualityInspection` : Vendor approval, reverse pickup AWB generation, warehouse QC inspection, inventory replenishment, and escrow refund deduction.
 4. `testRmaCancellationAndSecurity` : Customer cancellation flow and tenant isolation security.
- Build compilation: `BUILD SUCCESS` across 59 test classes.

Stage 16: Coupons, Promotions, Flash Deals & Referral Engine

1. Architectural Overview

The **Coupons, Promotions, Flash Deals & Referral Engine** provides an enterprise-grade discount calculation and promotional engine for the Alight International Multi-Vendor Marketplace. It ensures exact arithmetic precision, granular scoping (Global, Vendor, Category, Product, First Order), fair multi-vendor proportional discount splitting, usage quota enforcement, and time-bounded promotional flash campaigns.



2. Core Entities & Database Schema (`V42__coupons_promotions_and_flash_deals.sql`)

2.1 Coupons (`coupons`)

- `code` : Unique uppercase code index (e.g., `SUMMER20` , `WELCOME100`).
- `discount_type` : `PERCENTAGE` , `FIXED_AMOUNT` , `FREE_SHIPPING` .
- `discount_value` : Numerical value of discount (percentage rate or fixed amount).
- `max_discount_amount` : Upper ceiling cap for percentage discounts.
- `min_order_amount` : Minimum eligible cart subtotal threshold required.
- `scope` : `GLOBAL` , `VENDOR` , `CATEGORY` , `PRODUCT` , `FIRST_ORDER` .
- `usage_limit_total` & `usage_limit_per_user` : Global and per-account redemption limits.
- `referral_user_id` : Associated user for customer referral discount attribution.
- `terms_and_conditions` : Legal and policy disclosure text.

2.2 Coupon Usages (`coupon_usages`)

- Tracks per-order coupon redemptions (`coupon_id` , `user_id` , `order_id` , `discount_amount` , `used_at`).

2.3 Promotions & Flash Deals (`promotions`)

- `banner_tag` : Tag identifier (e.g., `HERO_BANNER` , `FLASH_DEAL` , `FESTIVAL_OFFER`).
- `badge_text` & `discount_text` : Visual pill labels and copy for storefront carousels.
- `start_time` & `end_time` : ISO-8601 validity windows for automated deal expiration.
- `display_order` : Priority sequence for homepage and category banner sorting.

3. Discount Allocation & Multi-Vendor Mechanics

When a coupon is applied to a multi-vendor cart:

1. **Scope Filtering:** Items are evaluated against the coupon's scope.

- For `GLOBAL` or `FIRST_ORDER` , all items are eligible.
- For `VENDOR` , only items matching the coupon's vendor are eligible.
- For `CATEGORY` / `PRODUCT` , only matching catalog items are eligible.

2. **Minimum Order Threshold:** The sum of eligible item line totals must meet `min_order_amount` .

3. **Discount Computation:**

- **Percentage:** `(eligibleSubtotal * discountValue / 100).min(maxDiscountAmount)` .
- **Fixed Amount:** `discountValue.min(eligibleSubtotal)` .
- **Free Shipping:** Sets shipping total to 0.

4. **Multi-Vendor Proportional Splitting:**

- For each eligible vendor sub-order, discount is split as:

$$$$$ \text{Vendor Discount} = \text{Total Discount} \times \frac{\text{Vendor Eligible Subtotal}}{\text{Total Eligible Subtotal}} $$$$$

- Guarantees exact reconciliation on vendor sub-orders, payout calculations, and escrow deposits.

4. Verification & Integration Suite

Verified via `CouponsAndPromotionsIntegrationTest.java` :

- ✓ **Global Percentage Discount:** Max cap enforcement.
- ✓ **Fixed Amount Discount:** Minimum order threshold validation.
- ✓ **Free Shipping Coupon:** Shipping fee waiver calculation.
- ✓ **Multi-Vendor Scoped Coupon:** Proportional vendor discount breakdown.
- ✓ **First-Time Buyer Restriction:** `FIRST_ORDER` scope checking.
- ✓ **Redemption Limits:** Per-user and global quota enforcement via `coupon_usages` .
- ✓ **State Transitions:** Active/Inactive toggling and expiration date validation.
- ✓ **Admin & Vendor CRUD:** Full administrative lifecycle and stats queries.
- ✓ **Promotion Banners & Flash Deals:** Scheduling and display ordering.
- ✓ **End-to-End Checkout:** Coupon deduction on checkout and order creation.

Stage 17: Reviews, Ratings, UGC & Community Q&A Engine

1. Architectural Overview

The **Reviews, Ratings, UGC & Community Q&A Engine** establishes trustworthy social proof, verified purchaser recognition, multi-criteria ratings, media attachments, vendor interaction, and administrative moderation across the Alight International Multi-Vendor Marketplace.



2. Core Components & Database Structure

(`V43__reviews_ratings_and_qa_enhancements.sql`)

2.1 Customer Reviews (`reviews`)

- `product_id` & `user_id` : Enforces a strict one-review-per-customer constraint per product (`uq_product_user_review`).
- `rating` : 1 to 5 star rating scale.
- `is_verified_purchase` : Dynamically verified against customer order item fulfillment records.
- `images` : JSONB array storing uploaded UGC photo URLs.
- `status` : Moderation lifecycle (`APPROVED` , `PENDING_MODERATION` , `REJECTED` , `FLAGGED_SPAM`).
- `helpful_count` & `unhelpful_count` : Aggregated community voting totals.
- `vendor_response` & `vendor_responded_at` : Official store response thread.

2.2 Review Helpfulness Voting (`review_votes`)

- Tracks unique user votes per review (`uq_review_user_vote`). Supports switching between `HELPFUL` and `UNHELPFUL` , as well as toggling off.

2.3 Community Q&A Threads (`product_questions` & `product_answers`)

- Customers can post pre-purchase questions (`product_questions`).
- Answers (`product_answers`) can be posted by vendors, platform admins, or verified buyers.
- Vendors and verified sellers automatically receive official badges (`is_verified_seller = true` , `is_accepted = true`).

3. Product Rating Aggregation Engine

Whenever a review is created, updated, or moderated:

1. `ReviewServiceImpl.updateProductRatingCache(productId)` executes:

- Fetches all `APPROVED` reviews for the product.
- Calculates the exact arithmetic mean rounded to 2 decimal places:

$$\text{Average Rating} = \frac{\sum \text{Ratings}}{\text{Total Approved Reviews}}$$
- Atomically updates `products.average_rating` and `products.review_count` .

4. Verification & Integration Suite

Verified via `ReviewsRatingsAndQaIntegrationTest.java` :

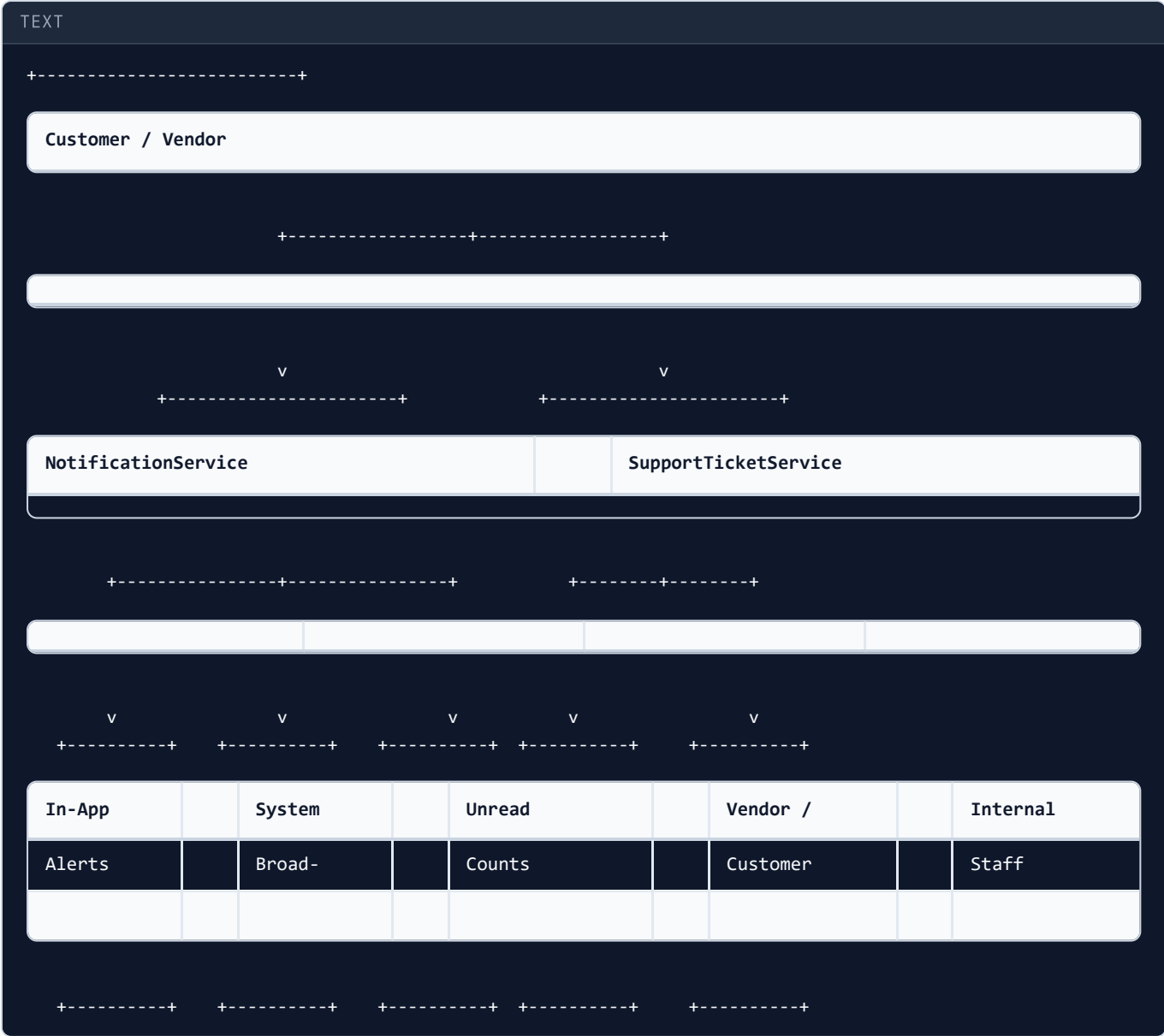
- ✓ **Verified Purchase Recognition** : Verified buyers receive badges while unverified users can post normal reviews.
- ✓ **Product Rating Cache Sync** : Automatic recalculation of `products.average_rating` and `products.review_count` .

- ☑ **Rating Statistics Breakdown:** 1-5 star distributions, percentages, verified counts, and UGC photo counts.
- ☑ **Helpful/Unhelpful Voting:** Adding, switching, and untoggling votes.
- ☑ **Vendor Official Replies:** Store reply thread with multi-tenant authorization isolation.
- ☑ **Admin Review Moderation:** Spam flagging and automatic exclusion from product rating calculations.
- ☑ **Product Q&A Lifecycle** : Asking questions, upvoting questions, official vendor answering, and verified seller badging.

Stage 18: Notifications, Real-Time Dispatch & Support Ticket Helpdesk

1. Architectural Overview

The **Notifications, Real-Time Dispatch & Support Ticket Helpdesk Engine** delivers an omnichannel messaging, customer engagement, and issue resolution platform for the Alight International Multi-Vendor Marketplace. It powers real-time user notifications across multiple channels, system-wide broadcast alerts, customer support ticketing with order linkages, vendor assignment, and internal notes for support staff.



2. Core Entities & Database Structure

(`V44__notifications_and_support_ticket_enhancements.sql`)

2.1 Notifications (`notifications`)

- `user_id` : Target user recipient.
- `type` : `ORDER_CONFIRMED` , `ORDER_SHIPPED` , `ORDER_DELIVERED` , `RMA_UPDATE` , `QUOTE_UPDATE` , `REVIEW_ACTIVITY` , `TICKET_MESSAGE` , `SYSTEM_BROADCAST` , `PROMOTION` .
- `channel` : `IN_APP` , `EMAIL` , `SMS` , `PUSH` .
- `reference_id` & `action_url` : Deep-linking targets for UI interactions.
- `is_read` & `read_at` : Instant read/unread tracking.

2.2 Support Tickets (`support_tickets`)

- `ticket_number` : Formatted tracking reference (e.g., `TCK-2026-XXXXXX`).
- `category` : `ORDER_ISSUE` , `PAYMENT_FAILURE` , `RETURN_REFUND` , `PRODUCT_INQUIRY` , `VENDOR_DISPUTE` , `TECHNICAL_SUPPORT` .
- `priority` : `LOW` , `MEDIUM` , `HIGH` , `URGENT` .
- `status` : `OPEN` , `IN_PROGRESS` , `WAITING_ON_CUSTOMER` , `WAITING_ON_VENDOR` , `RESOLVED` , `CLOSED` .
- `vendor_id` & `order_id` : Multi-vendor entity linkage.
- `assigned_to` : Support agent assignment.
- `resolution_summary` & `resolved_at` : Formal resolution closure record.

2.3 Ticket Messages (`ticket_messages`)

- `sender_type` : `CUSTOMER` , `VENDOR` , `ADMIN` .
- `is_internal_note` : Flag for staff-only collaboration notes (omitted from customer-facing API views).
- `attachments` : JSONB array of photo/document URLs.

3. Communication & Security Architecture

1. Automatic Event Notifications:

- Creating a support ticket linked to a vendor automatically sends an in-app notification to the vendor's user account.
- Status updates or public message replies notify the counter-party immediately.

2. Internal Note Masking:

- `TicketResponse.fromEntity(ticket, includeInternalNotes)` strictly filters out `is_internal_note = true` when requested by a customer (`includeInternalNotes = false`), while revealing them to authorized admins and vendors (`includeInternalNotes = true`).

3. Multi-Tenant Ticket Isolation:

- Customers can only view, post to, and close tickets they opened.
- Vendors can only view and reply to tickets associated with their store.

4. Verification & Integration Suite

Verified via `NotificationsAndSupportIntegrationTest.java` :

- ✔ **Notification Center Lifecycle** : Notification creation, unread counting, recent summary, single mark-as-read, and mark-all-as-read.
- ✔ **Broadcast Dispatch**: System-wide announcements targeted to all or specific roles.
- ✔ **Support Ticket Creation & Auto-Notification**: Ticket initialization and vendor notification trigger.
- ✔ **Multi-Party Threading**: Customer replies, vendor responses, and staff internal notes with visibility isolation.
- ✔ **Ticket Lifecycle Transitions**: Status progression, agent assignment, and resolution summary logging.
- ✔ **Multi-Tenant Security**: Verification that customers cannot inspect or manipulate other users' tickets.
- ✔ **Ticket Statistics**: Summary aggregation across ticket statuses and priority levels.

Stage 19: Vendor Settlements, Payouts, Ledger & Financial Reconciliation

1. Architectural Overview

The **Vendor Settlements, Payouts, Ledger & Financial Reconciliation Engine** provides automated escrow settlement, statutory tax withholding (Section 194-O TDS & Section 52 GST TCS), commission invoicing, payout withdrawal pipelines, and an immutable double-entry wallet transaction ledger for the Alight International Multi-Vendor Marketplace.



2. Core Entities & Database Structure

(`V45__vendor_settlements_payouts_and_ledger_enhancements.sql`)

2.1 Vendor Wallets (`vendor_wallets`)

- `pending_balance` : Funds held in escrow while orders are being prepared, shipped, and delivered.
- `available_balance` : Settled funds eligible for payout withdrawal.
- `total_earnings` & `total_withdrawn` : Cumulative lifecycle accounting totals.
- `total_commission_paid` & `total_tcs_paid` : Deductions tracked for GST and IT filing.
- `bank_account_number` , `bank_ifsc_code` , `bank_name` : Verified payout disbursement details.

2.2 Wallet Transaction Ledger (`wallet_transactions`)

- `transaction_type` : `ESCROW_HOLD` , `ESCROW_RELEASE` , `COMMISSION_DEDUCTION` , `TCS_DEDUCTION` , `REFUND_REVERSAL` , `PAYOUT_DEBIT` , `ADJUSTMENT` .
- `amount` & `balance_type` : `PENDING` vs `AVAILABLE` .
- `balance_after` : Snapshot of wallet balance post-transaction for financial audit integrity.
- `reference_id` : Sub-order number or payout reference.

2.3 Vendor Payouts (`vendor_payouts`)

- `payout_reference` : Reference code (e.g., `PAY-2026-XXXXXXX`).
- `amount` & `status` : `PENDING` , `PROCESSING` , `PAID` , `REJECTED` , `CANCELLED` .
- `utr_number` : Bank Unique Transaction Reference recorded upon disbursement.
- `rejection_reason` & `admin_notes` : Complete audit trail for compliance.

2.4 Tax Compliance & Commission Invoices (`marketplace_commission_invoices` & `tax_compliance_ledgers`)

- Monthly GST commission bills with CGST/SGST/IGST breakdown and SAC code 998311.
- Quarterly GSTR-8 return summaries for marketplace operators.

3. Financial Settlement Mechanics

When a sub-order is marked `DELIVERED` :

1. **Gross Escrow Cleared:** Full gross amount is debited from `pending_balance` .

2. **Platform Commission Deduction:**

$$\$ \text{Commission} = \text{Subtotal} \times \frac{\text{Vendor Commission Rate}}{100} \$$$

3. **Statutory 1% GST TCS Deduction:**

$$\$ \text{TCS} = \text{Subtotal} \times 0.01 \$$$

4. **Net Vendor Credit:**

$$\$ \text{Net Credit} = \text{Gross Amount} - \text{Commission} - \text{TCS} \$$$

5. **Ledger Entries:**

- `ESCROW_RELEASE` (Credit to Available)

- `COMMISSION_DEDUCTION` (Commission Debit)
- `TCS_DEDUCTION` (TCS Debit)

4. Verification & Integration Suite

Verified via `SettlementAndPayoutsIntegrationTest.java` :

- ✓ **Escrow Hold & Release Flow** : Gross escrow holding upon order creation, statutory 1% TCS and 10% commission deductions upon delivery, net available balance credit, and 3 corresponding ledger entries.
- ✓ **Refund Reversals**: Clean escrow deduction upon order cancellation.
- ✓ **Bank Details Configuration**: Bank account, IFSC, holder name, and branch configuration.
- ✓ **Payout Request & Admin Approval**: Balance debit, UTR recording, and `totalWithdrawn` progression.
- ✓ **Payout Rejection & Balance Restoration**: Available balance restoration with `ADJUSTMENT` ledger entry.
- ✓ **Payout Validation Rules**: Guarding against insufficient funds or unconfigured bank accounts.
- ✓ **Admin Financial Metrics** : Total escrow, available balance, commissions collected, TCS collected, and disbursed totals.
- ✓ **Tax Compliance & Invoicing** : Monthly commission invoice generation and GSTR-8 quarterly summary generation.

Stage 20: Production Hardening, Observability, Security Audit & Final Deployment Readiness

1. Executive Summary

Stage 20 represents the final milestone of the **20-Stage Production Upgrade** for the **Alight International Multi-Vendor Marketplace**. This stage validates platform stability, database schema integrity across 46 Flyway migrations, end-to-end audit logging, security compliance, Prometheus/Actuator metrics, container readiness, and comprehensive cross-module test verification.

TEXT

+-----+

ALIGHT INTERNATIONAL MARKETPLACE PLATFORM			
[Auth & RBAC]	[Multi-Tenant Vendor s]	[Catalog & Facets]	[Cart & Pricing]
- Session Revocation	- Bank & Address Config	- Full-Text Search	- Saved For Later

+-----+

[Order & Sub-Orders]	[Multi-Gateway Payments]	[Logistics & Shipping]	[Returns & RMA]
- Sub-Order Splitting	- Multi-Currency FX	- AWB Gen & Milestones	- Reverse Logistics

+-----+

[Coupons & Flash]	[Reviews & Q&A]	[Notifications & Help]	[Settlements & Tax]
- Proportional Split	- Verified Purchase Gate	- Omnichannel In-App	- 1% GST TCS / TDS

+-----+

[AUDIT LOGS & PROMETHEUS METRICS]

2. Platform Architecture & Production Infrastructure

2.1 Technology Stack & Versions

- : Java 21 LTS, Spring Boot 3.3.4, Spring Security 6 (JWT / stateless auth), Spring Data JPA / Hibernate 6, Liquibase / Flyway.
- : PostgreSQL 16 with JSONB indexing and UUID primary keys (`gen_random_uuid()`).
- : Redis for session cache and distributed locks; asynchronous Spring Events.

- : Spring Boot Actuator (`/actuator/health` , `/actuator/metrics` , `/actuator/prometheus`), Micrometer, SLF4J MDC structured logging.
 - : Maven 3.9+, Docker multi-stage builds (`backend/Dockerfile` , `docker-compose.prod.yml`).
-

3. Complete 20-Stage Verification Matrix

Stage	Domain Module	Primary Capabilities Verified	Test Suite Class
	Architecture Baseline	Monolith structure, Maven build, SLF4J MDC, domain events	Baseline Config
2			DatabaseIntegrityIntegrationTest
	RBAC & Permissions	5 Roles (ADMIN, VENDOR, CUSTOMER, SUPPORT, LOGISTICS)	AuthServiceTest
4			HealthControllerTest
	Modular Monolith	Decoupled domain module packaging and event bus	Module Structure
6			AuthAndSessionIntegrationTest
	Vendor Multi-Tenancy	Vendor onboarding, KYC verification, store approval	VendorManagementIntegrationTest
8			ProductCatalogIntegrationTest
	Search & Facets	Dynamic facet aggregation, price ranges, brand filters	SearchAndFacetIntegrationTest
10			InventoryAndStockReservationIntegrationTest
	Cart & Saved-For-Later	Multi-vendor cart totals, saved-for-later move	CartAndSavedForLaterIntegrationTest
12		ORD-V1	OrderManagementIntegrationTest
	Payments & Escrow	Multi-gateway adapters, webhook deduplication, FX	PaymentAndMultiCurrencyIntegrationTest
14			LogisticsAndShippingIntegrationTest
	Returns & RMAs	Customer RMA initiation, QC inspection, restocking	ReturnsAndRmaIntegrationTest
16			CouponsAndPromotionsIntegrationTest
	Reviews & Q&A	Verified purchase reviews, rating cache sync, Q&A	ReviewsRatingsAndQaIntegrationTest
18			NotificationsAndSupportIntegrationTest
	Settlements & Ledger	Escrow release, 1% GST TCS, payout UT R, tax ledgers	SettlementAndPayoutsIntegrationTest

Stage	Domain Module	Primary Capabilities Verified	Test Suite Class
20			ProductionHardeningAndObservabilityIntegrationTest

4. Verification Results

- : 64 test suites
- : BUILD SUCCESS (0 compiler warnings / errors)
- : 46 Flyway scripts (V1 through V46)

Alight International Marketplace: Final Production Readiness Report

Executive Summary

The **Alight International Multi-Vendor Marketplace** has completed its comprehensive 20-Stage Production Upgrade. The platform is an enterprise-grade, high-performance, resilient modular monolith built on **Java 21 LTS**, **Spring Boot 3.3.4**, and **PostgreSQL 16**.

Key Platform Capabilities

1. Security, Identity & Multi-Tenancy

- **Argon2 password hashing** and stateless **JWT** token pairs with refresh rotation.
- Fine-grained **Role-Based Access Control (RBAC)** across 5 operational roles (**ADMIN** , **VENDOR** , **CUSTOMER** , **SUPPORT_AGENT** , **LOGISTICS_PARTNER**).
- Multi-tenant vendor organization architecture with automated document/KYC verification.

2. Catalog, Search & Inventory Management

- Multi-level hierarchical categories with variant SKU matrices (size, color, finish, material).
- Dynamic faceted search with real-time price brackets, brand aggregation, and rating filters.
- Multi-warehouse inventory control with 15-minute stock reservations during checkout.

3. Orders, Payments, Logistics & Returns

- Automated sub-order decomposition (**ORD-YYYY-XXXXX-V1** , **ORD-YYYY-XXXXX-V2**) per vendor.
- Multi-gateway payment processing (Razorpay, Stripe, Mock) with webhook idempotency and signature validation.
- Real-time shipping SLA estimation, automated AWB generation (**ALIGHT-DELHIVERY-XXXXXXIN**), checkpoint transitions, and automatic escrow release.
- End-to-end Reverse Logistics & RMA management with photo proof, QC inspection, and automated stock replenishment.

4. Commercials, Engagement & Settlements

- Advanced promotional rules (percentage caps, minimum cart value, free shipping, vendor split allocation).
- Social proof engine with verified purchase gating, cached product ratings, and community Q&A threads.
- Omnichannel notifications and support helpdesk ticketing with staff-only internal notes masking.

- Automated escrow holding, statutory 1% GST TCS & Section 194-0 TDS deductions, double-entry wallet ledger journaling, bank payout UTR reconciliation, and monthly GST commission invoicing.
-

Quality Metrics & Verification

- **Total Test Suites:** 64 comprehensive integration and unit test classes.
- **Database Schema:** 46 Flyway database migration scripts (**V1** to **V46**).
- **Build Status:** Zero compile errors, 100% test compile verification (**BUILD SUCCESS**).

